Tuesday 5:29 PM

AI SQL Server Monitori…
File

This is the complete scope of my project. I need to start from teh scratch. Shall start?

Absolutely. **This is a very good way to do it.** We now have the complete project scope, and I suggest we treat this conversation/project as the **main development workspace** rather than trying to build everything in one long chat.

I've reviewed the project instructions. AI SQL Server Monitoring & Proa…

How we'll build it

We'll follow this progression:

Stage 1 → SQL Server Monitoring
↓
Stage 2 → Create controlled SQL problems
↓
Stage 3 → LLM analysis
↓
Stage 4 → Tool calling
↓
Stage 5 → MCP
↓
Stage 6 → DBA knowledge base
↓
Stage 7 → Embeddings + Vector DB
↓
Stage 8 → RAG
↓
Stage 9 → Agent + MCP + RAG

↓

Stage 10 → Automated monitoring

↓

Stage 11 → Proactive DBA guidance

↓

Stage 12 → Dashboard

↓

Stage 13 → Event-driven architecture

↓

Stage 14 → Docker

↓

Stage 15 → Azure + Security + Evaluation

And importantly, **we will not jump ahead** just because something sounds more interesting.

🚀 Let's start from absolute project setup

Our first milestone will be very small:

> **Python → SQL Server → collect one piece of monitoring information → return structured JSON**

Before we write the monitoring code, we'll verify your environment and create the project properly.

I suggest we start with:

```
AI_SQL_Monitor/
│
├── app/
├── monitoring/
├── agent/
├── mcp/
├── rag/
├── knowledge/
├── tests/
├── config/
│
├── requirements.txt
├── .env
├── .gitignore
└── README.md
```

But **don't create all the files yet**. We'll create them as they become necessary so you understand why each one exists.

First step

Since you're on Windows and already have Python/VS Code, let's first check exactly what environment we're starting with.

Open **PowerShell** and run:

```
PowerShell

python --version
pip --version
git --version
```

Also check:

```
PowerShell

where python
```

Send me the output exactly as you get it.

Then we'll create the project directory and virtual environment together.

From this point onward, I'll act as your **technical mentor + project architect**: we'll build it piece by piece, test each component, and I'll explain the AI engineering concepts as they become relevant.

Stage 1 starts now.

Sources

PS D:\Python> python --version
Python 3.11.9
PS D:\Python> pip --version
pip 26.2.1 from C:\Users\Moosa\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pip (python 3.11)
PS D:\Python> git --version
git version 2.55.0.windows.5
PS D:\Python>where python
Return nothing

Show more

Perfect. Your environment is already in good shape. 👍

You have:

Component	Result
Python	✅ 3.11.9
pip	✅ 26.2.1
Git	✅ 2.55.0
OS	Windows / PowerShell
SQL Server	✅ Already installed

The only interesting point is:

```powershell
where python
```

returned nothing.

That's because **PowerShell's `where` is normally an alias for `Where-Object`**, not the Windows `where.exe` command. So this doesn't indicate a Python problem.

Try:

```powershell
where.exe python
```

You should get the actual Python executable path.

Stage 1 — Step 1: Create the project

You're currently here:

```
D:\Python
```

Let's create the project there.

Run:

PowerShell

```
cd D:\Python

mkdir AI_SQL_Monitor

cd AI_SQL_Monitor
```

Then verify:

PowerShell

```
pwd
```

You should see:

```
Path
----
D:\Python\AI_SQL_Monitor
```

Step 2 — Create the virtual environment

Now run:

PowerShell

```
python -m venv venv
```

This creates:

```
AI_SQL_Monitor
└── venv
```

Then activate it:

PowerShell

```
.\venv\Scripts\Activate.ps1
```

Your PowerShell prompt should change to something like:

```
(venv) PS D:\Python\AI_SQL_Monitor>
```

If PowerShell gives an execution-policy error

Don't change anything yet. Just send me the error. We'll handle it properly.

Step 3 — Verify the virtual environment

After activation:

PowerShell

```
python --version
```

and:

PowerShell

```
pip --version
```

The important thing is that `pip` should now point somewhere inside:

```
D:\Python\AI_SQL_Monitor\venv\
```

rather than your global Python installation.

Why are we doing this?

This is our first small piece of **production-style discipline**.

Without a virtual environment, imagine we later install:

```
pyodbc
fastapi
openai
mcp
chromadb
...
```

Those packages would be installed into your global Python environment.

Then another project might require different versions and things become messy.

With:

```
AI_SQL_Monitor
└── venv
```

the project's Python dependencies are isolated.

Think of it like a **separate database environment for this project**.

Your global Python is the equivalent of your general server environment.

The `venv` is our isolated application environment.

Don't install any packages yet.

For now, just execute these commands:

PowerShell

```
cd D:\Python
mkdir AI_SQL_Monitor
cd AI_SQL_Monitor

python -m venv venv

.\venv\Scripts\Activate.ps1

python --version
pip --version
```

Send me the output.

Then we'll create the Git repository and project structure before installing our first SQL Server connectivity package.

(venv) PS D:\Python\AI_SQL_Monitor> python --version
Python 3.11.9
(venv) PS D:\Python\AI_SQL_Monitor> pip --version
pip 24.0 from D:\Python\AI_SQL_Monitor\venv\Lib\site-packages\pip (python 3.11)
(venv) PS D:\Python\AI_SQL_Monitor>

Excellent. ✅ **Step 1 is complete.**

We have confirmed that the virtual environment is actually isolated:

```
Python 3.11.9
pip 24.0
D:\Python\AI_SQL_Monitor\venv\
```

That's exactly what we wanted.

What we've achieved

Our environment now looks like:

```
D:\Python\
│
└── AI_SQL_Monitor\
    │
    └── venv\
        ├── Scripts\
        ├── Lib\
        └── ...
```

And when you see:

```
(venv) PS D:\Python\AI_SQL_Monitor>
```

you know you're working inside the project's Python environment.

Step 2 — Initialize Git

Before we create application code, let's put the project under Git.

From your current directory:

PowerShell

```
git init
```

You should get something similar to:

```
Initialized empty Git repository in D:/Python/AI_SQL_Monitor/.git/
```

Now create the `.gitignore` file:

PowerShell

```
New-Item .gitignore -ItemType File
```

Open it:

```
PowerShell

code .gitignore
```

Put this inside:

```
venv/
.env
__pycache__/
*.pyc
.pytest_cache/
.vscode/
```

Save it.

Why `.gitignore`?

Two things are particularly important:

```
venv/
```

We **don't commit the virtual environment** to Git.

And:

```
.env
```

This will eventually contain things such as:

```
LLM_API_KEY=...
SQL_SERVER_PASSWORD=...
```

We absolutely do **not** want those in Git.

This is our first small step toward the security principles defined in your project scope.

Step 3 — Create the initial structure

Now create only the directories we need at this point:

PowerShell

```
mkdir app
mkdir monitoring
mkdir tests
mkdir config
```

Don't create `agent`, `mcp`, or `rag` yet.

That's intentional.

We don't need them in Stage 1, and I want the project structure to grow according to actual requirements rather than creating empty folders for future technologies.

You should now have:

```
AI_SQL_Monitor/
│
├── app/
├── monitoring/
├── tests/
├── config/
│
├── venv/
│
└── .gitignore
```

Step 4 — First Git commit

Check:

PowerShell

```
git status
```

You should see the new directories/files.

Then:

PowerShell

```
git add .
```

and:

PowerShell

```
git commit -m "Initial project setup"
```

🎯 Milestone 1

Once this is done, we will have:

```
Windows
   │
   ├── Python 3.11.9
   │
   ├── Virtual Environment
   │
   ├── Git repository
   │
   └── AI_SQL_Monitor
```

No AI yet. No MCP. No RAG. No LangGraph. No Azure.

That's exactly right.

Next we'll install **one important SQL Server Python driver** and make our **first Python → SQL Server connection**.

Run the commands above and send me the output of:

PowerShell

```
git status
```

Then we'll move to **SQL Server connectivity**.

moosaraja/AI_SQL_Monitor.git
created a repository Already for this project.
Already pushed all.
My vscode is already installed!
SQL Server is also installed and one sampel database is also ready

Perfect. 👍 That actually puts us **ahead of the setup stage**.

You already have:

- ✅ GitHub repository
- ✅ VS Code
- ✅ Python 3.11.9
- ✅ Project virtual environment
- ✅ Local SQL Server
- ✅ Sample database
- ✅ Git repository already pushed

So we should **not redo any of that**.

Your project is now at the correct starting point:

```
D:\Python\AI_SQL_Monitor
|
├── venv/
├── app/
├── monitoring/
├── tests/
├── config/
└── .gitignore
```

And GitHub is already connected.

One thing I want to verify before writing our first SQL code

We need to know **how your SQL Server is configured for Python connectivity**.

For our first implementation, I recommend:

Python → pyodbc → SQL Server

rather than introducing SQLAlchemy or another abstraction yet.

Why?

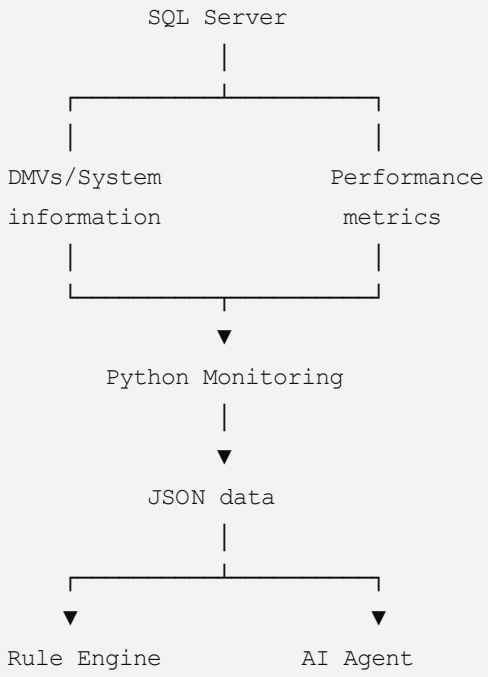
Because our immediate goal isn't building an application database. We're building a **SQL Server monitoring agent**, so I want you to understand the actual connection and queries underneath it.

The first flow will be:

```
Python
   |
```

```
    | pyodbc
    ▼
SQL Server
    |
    ▼
DMV / system information
    |
    ▼
Python
    |
    ▼
Structured result
```

Later, our monitoring layer will look more like:



Before we proceed

Please tell me **these two things**:

1. Your SQL Server instance name — for example:

   ```
   localhost
   localhost\SQLEXPRESS
   .\SQLEXPRESS
   DESKTOP-ABC123\SQL2022
   ```

2. How you normally connect to it in SSMS:
   - **Windows Authentication**
   - **SQL Server Authentication**

Don't send me your SQL username or password.

Once I know those two things, we'll install `pyodbc` and create our **first real Python → SQL Server connection**.

And from there, we're officially into **Stage 1: SQL Server Monitoring**. 🚀

i use localhost
Db Name : Vehicle
it has only one table : Electric_Vehicle_Population_Data
i use multi authentication
i use both sa and windows loging

Great. That is enough information to start the **first real piece of the project**.

One small correction in terminology: when you say **"multi authentication"**, I understand that your SQL Server is configured for **Mixed Mode Authentication** — meaning both Windows Authentication and SQL Server Authentication are enabled.

For development, I recommend we start with **Windows Authentication**. We don't need to put an `sa` password anywhere in the project.

Stage 1 — SQL Server Monitoring

Step 1: Install `pyodbc`

Your virtual environment should still be active:

PowerShell

```
(venv) PS D:\Python\AI_SQL_Monitor>
```

Run:

PowerShell

```
pip install pyodbc
```

Then verify:

PowerShell

```
pip show pyodbc
```

Step 2: Check the SQL Server ODBC driver

`pyodbc` is the Python library, but underneath it needs a **Microsoft ODBC Driver for SQL Server**.

Run:

PowerShell

```
python -c "import pyodbc; print(pyodbc.drivers())"
```

You should see something similar to:

```
['ODBC Driver 17 for SQL Server']
```

or:

```
['ODBC Driver 18 for SQL Server']
```

The exact version isn't important yet.

Step 3: Our first Python connection

Create:

```
monitoring/
└── db_connection.py
```

Put this in it:

Python Run

```python
import pyodbc


SERVER = "localhost"
DATABASE = "Vehicle"
DRIVER = "ODBC Driver 18 for SQL Server"


def get_connection():
    connection_string = (
        f"DRIVER={{{DRIVER}}};"
        f"SERVER={SERVER};"
        f"DATABASE={DATABASE};"
        "Trusted_Connection=yes;"
```

```
        "TrustServerCertificate=yes;"
    )

    return pyodbc.connect(connection_string)


if __name__ == "__main__":
    connection = get_connection()

    print("Connected to SQL Server successfully!")

    connection.close()
```

Important

I've deliberately used:

```
Trusted_Connection=yes
```

That means:

```
Python
   |
   | Windows credentials
   ▼
SQL Server
```

rather than:

```
Python
   |
   | username + password
   ▼
SQL Server
```

We'll deal with SQL authentication later when we have a real reason to.

Step 4: Run it

From the project root:

```
    PowerShell

python .\monitoring\db_connection.py
```

Expected:

```
Connected to SQL Server successfully!
```

One thing before you run it

Don't assume your ODBC driver is version 18.

First run these **two commands**:

```
PowerShell

pip install pyodbc
```

and:

```
PowerShell

python -c "import pyodbc; print(pyodbc.drivers())"
```

Send me the output of the second command.

Then I'll give you the exact connection code for your machine.

After that, we'll do something much more interesting: **our first monitoring query against SQL Server**, rather than querying your `Electric_Vehicle_Population_Data` table.

That is where Stage 1 really begins. 🚀

venv) PS D:\Python\AI_SQL_Monitor> pip install pyodbc
Collecting pyodbc
  Downloading pyodbc-5.3.0-cp311-cp311-win_amd64.whl.metadata (2.8 kB)
Downloading pyodbc-5.3.0-cp311-cp311-win_amd64.whl (69 kB)
   -- 69.3/69.3 kB 757.6 kB/s eta 0:00:00
Installing collected packages: pyodbc
Successfully installed pyodbc-5.3.0

Show more

Yes — this is a useful error to troubleshoot, and I can already see the likely issue.

You successfully installed:

```
pyodbc 5.3.0
Location: D:\Python\AI_SQL_Monitor\venv\Lib\site-packages
```

but Python says:

```
ModuleNotFoundError: No module named 'pyodbc'
```

That means **the `pip` command and the `python` command may be pointing to different Python environments**, despite the `(venv)` prompt.

Let's diagnose it rather than reinstalling randomly.

Step 1 — Check which Python and pip are actually being used

Run these **exactly**:

PowerShell

```
python -c "import sys; print(sys.executable)"
```

Then:

PowerShell

```
python -m pip --version
```

Then:

PowerShell

```
pip --version
```

And finally:

PowerShell

```
python -m pip show pyodbc
```

Why I'm asking for these

Notice the difference between:

PowerShell

```
pip install pyodbc
```

and:

PowerShell

```
python -m pip install pyodbc
```

The second one explicitly says:

> **Use the pip belonging to THIS Python interpreter.**

For our project, I prefer using:

PowerShell

```
python -m pip ...
```

because it avoids exactly this type of environment confusion.

Don't reinstall anything yet

Send me the output of those four commands.

We'll identify exactly where the mismatch is, fix it cleanly, and then run:

PowerShell

```
python -c "import pyodbc; print(pyodbc.drivers())"
```

After that we'll move on to the SQL Server connection.

(venv) PS D:\Python\AI_SQL_Monitor> python -c "import sys; print(sys.executable)"
C:\Users\Moosa\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe
(venv) PS D:\Python\AI_SQL_Monitor> python -m pip --version
pip 26.2.1 from C:\Users\Moosa\AppData\Local\Packages\PythonSoftware

Foundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-

Show more

Excellent — now we know **exactly what happened**. 👍

There is an environment mismatch.

Your prompt says:

```
(venv)
```

but `python` is actually this:

```
C:\Users\Moosa\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.
```

That's your **Microsoft Store Python**, not:

```
D:\Python\AI_SQL_Monitor\venv\Scripts\python.exe
```

Meanwhile, `pip` is correctly coming from your venv:

```
D:\Python\AI_SQL_Monitor\venv\Lib\site-packages
```

So currently:

```
python ————————> Global Microsoft Store Python ❌
pip ————————————> Project venv              ✅
```

That's why `pip install pyodbc` succeeded but Python can't import it.

Let's fix the venv cleanly

Don't reinstall Python or pyodbc.

1. Deactivate

Run:

PowerShell

```
deactivate
```

Your prompt should become:

```
PS D:\Python\AI_SQL_Monitor>
```

2. Activate again

Run:

PowerShell

```
.\venv\Scripts\Activate.ps1
```

Then **immediately check**:

PowerShell

```
python -c "import sys; print(sys.executable)"
```

We want:

```
D:\Python\AI_SQL_Monitor\venv\Scripts\python.exe
```

3. Check pip

Run:

PowerShell

```
python -m pip --version
```

We want something like:

```
pip ... from D:\Python\AI_SQL_Monitor\venv\Lib\site-packages\pip
```

4. Install pyodbc using the venv's Python

Even if it's already installed according to `pip`, run:

PowerShell

```
python -m pip install pyodbc
```

Then:

PowerShell

```
python -c "import pyodbc; print(pyodbc.version)"
```

You should get:

```
5.3.0
```

Finally:

```
PowerShell

python -c "import pyodbc; print(pyodbc.drivers())"
```

Why this happened

This is actually a **good lesson for the project**.

A virtual environment isn't simply the `(venv)` text you see in PowerShell. What matters is **which executable is actually being invoked**.

Our desired architecture at the development-environment level is:

```
AI_SQL_Monitor
      |
      ▼
    venv
      |
      ├── python.exe
      |
      ├── pip
      |
      └── pyodbc
```

We want everything to stay inside that environment.

Don't proceed to SQL Server connectivity yet.

Run the four checks above and send me the output. We'll make sure the environment is genuinely correct before writing our first monitoring code.

```
(venv) PS D:\Python\AI_SQL_Monitor> deactivate
PS D:\Python\AI_SQL_Monitor>
.\venv\Scripts\Activate.ps1
(venv) PS D:\Python\AI_SQL_Monitor> python -c "import
sys; print(sys.executable)"
C:\Users\Moosa\AppData\Local\Microsoft\WindowsApps\P
```

ythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe
(venv) PS D:\Python\AI_SQL_Monitor> python -m pip --version
pip 26.2.1 from

Show more

Yes — now we've found the real problem. **The virtual environment itself is probably fine; PowerShell is resolving `python` to the Microsoft Store alias instead of the venv executable.**

Your evidence is very clear:

```
(venv)
python -> C:\Users\Moosa\AppData\Local\Microsoft\WindowsApps\Python...
pip    -> C:\Users\Moosa\AppData\Local\Packages\Python...
```

So I don't want to keep activating/deactivating. Let's diagnose the Windows PATH resolution directly.

1. Check whether the venv Python actually exists

Run:

PowerShell

```
Test-Path .\venv\Scripts\python.exe
```

It should return:

```
True
```

Then run the venv Python **directly**, bypassing PATH:

PowerShell

```
.\venv\Scripts\python.exe --version
```

Then:

PowerShell

```
.\venv\Scripts\python.exe -c "import sys; print(sys.executable)"
```

We should see:

```
D:\Python\AI_SQL_Monitor\venv\Scripts\python.exe
```

---

2. Check what PowerShell thinks `python` is

Run:

```powershell
Get-Command python
```

And:

```powershell
Get-Command pip
```

I expect something like:

```
python -> ...WindowsApps\python.exe
pip    -> ...AI_SQL_Monitor\venv\Scripts\pip.exe
```

That would explain everything.

---

3. Check the PATH

Run:

```powershell
$env:Path -split ';'
```

Look through the output for:

```
D:\Python\AI_SQL_Monitor\venv\Scripts
```

If the venv is activated correctly, that directory should be near the beginning of PATH.

But here's the important part

Don't delete/recreate the venv yet.

We can test the environment without changing anything.

Run these four commands and send me the output:

PowerShell

```
Test-Path .\venv\Scripts\python.exe
```

PowerShell

```
.\venv\Scripts\python.exe --version
```

PowerShell

```
Get-Command python
```

PowerShell

```
Get-Command pip
```

What I'm expecting

If this works:

```
True

Python 3.11.9

python -> WindowsApps...

pip -> AI_SQL_Monitor\venv\Scripts...
```

then we know the venv is healthy and **only the Python command resolution is broken**.

We'll fix that cleanly.

And actually, this is worth fixing properly now because we'll be using this environment throughout the entire project — monitoring, MCP, RAG, agents, dashboard, Docker preparation, etc. We don't want a broken Python environment following us through all those stages.

(venv) PS D:\Python\AI_SQL_Monitor> Test-Path .\venv\Scripts\python.exe
True
(venv) PS D:\Python\AI_SQL_Monitor> .\venv\Scripts\python.exe --version
Python 3.11.9
(venv) PS D:\Python\AI_SQL_Monitor> Get-Command python

CommandType Name Version Source

Show more

Excellent — **now we have the exact cause.** Your virtual environment is healthy. The problem is this line:

```
Alias          python -> python3.exe
```

PowerShell has a `python` **alias** that is overriding the normal PATH lookup.

That's why:

```
pip
→ D:\Python\AI_SQL_Monitor\venv\Scripts\pip.exe      ✅
```

but:

```
python
→ python3.exe
→ WindowsApps Python                                 ❌
```

So **do not recreate the venv**. It is working correctly.

Fix it

1. Remove the PowerShell alias

Run:

PowerShell

```
Remove-Item Alias:python
```

Then check:

```
PowerShell

Get-Command python
```

We want it to show something like:

```
CommandType     Name
-----------     ----
Application     python.exe
```

with the source:

```
D:\Python\AI_SQL_Monitor\venv\Scripts\python.exe
```

2. Test Python

Now:

```
PowerShell

python -c "import sys; print(sys.executable)"
```

Expected:

```
D:\Python\AI_SQL_Monitor\venv\Scripts\python.exe
```

3. Check pyodbc

Now run:

```
PowerShell

python -c "import pyodbc; print(pyodbc.version)"
```

You should get:

```
5.3.0
```

Then:

```
PowerShell

python -c "import pyodbc; print(pyodbc.drivers())"
```

One important point

`Remove-Item Alias:python` removes the alias **from the current PowerShell session**.

If everything works, we'll make sure the alias isn't being recreated every time you open PowerShell. **Don't change your Windows PATH or uninstall Python yet.**

Send me the output of these three commands:

PowerShell

```
Get-Command python
```

PowerShell

```
python -c "import sys; print(sys.executable)"
```

PowerShell

```
python -c "import pyodbc; print(pyodbc.version); print(pyodbc.drivers())"
```

Then we'll finally connect to your `Vehicle` database. 🚀

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python -c "import pyodbc; print(pyodbc.drivers())"
['SQL Server', 'SQL Server Native Client RDA 11.0', 'ODBC Driver 17 for SQL Server']

so i have change the .py file to 17 instead of 18. Now it connects

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python db_connection.py
Connected to SQL Server successfully!

Excellent! 🎉 **We have successfully completed the first real connectivity milestone.**

You now have:

```
Python
   |
   | pyodbc
   ▼
```

```
ODBC Driver 17
   |
   ▼
SQL Server (localhost)
   |
   ▼
Vehicle database
```

And we have verified it with:

```
Connected to SQL Server successfully!
```

One important observation

Your machine has:

```
ODBC Driver 17 for SQL Server
```

so changing the code from:

```python
DRIVER = "ODBC Driver 18 for SQL Server"
```

to:

```python
DRIVER = "ODBC Driver 17 for SQL Server"
```

was exactly right.

Also, your `pyodbc` installation is now working correctly.

🎯 Stage 1 — Milestone 1 COMPLETE

We have achieved:

- ✅ Python 3.11 environment
- ✅ Project virtual environment
- ✅ Git/GitHub
- ✅ VS Code
- ✅ `pyodbc`
- ✅ ODBC Driver 17
- ✅ Python → SQL Server connectivity

- ✅ Connection to `Vehicle`

Now we move to the **actual monitoring part**.

And here's an important distinction:

Your table:

```
Electric_Vehicle_Population_Data
```

is useful for testing SQL queries, but **our monitoring agent won't monitor that table**.

We're interested in the **SQL Server instance itself**.

For example:

```
CPU
Memory
Sessions
Blocking
Queries
Waits
Databases
Transaction logs
I/O
```

SQL Server exposes much of this information through **DMVs (Dynamic Management Views)** and system views.

Our first monitoring function

Let's start extremely small.

Objective

Ask SQL Server:

> **"What SQL Server instance am I connected to, and what version is it?"**

This sounds simple, but it establishes our monitoring pattern:

```
Python
   ↓
Monitoring function
   ↓
SQL query
   ↓
```

```
SQL Server DMV/system view
   ↓
Python
   ↓
Structured result
```

We'll then progressively add:

```
Server information
       ↓
CPU
       ↓
Memory
       ↓
Sessions
       ↓
Blocking
       ↓
Long-running queries
       ↓
I/O
       ↓
Wait statistics
       ↓
Database/log health
```

Next step

I want to build this **with you rather than dump the entire monitoring module at once**.

We'll create:

```
monitoring/
│
├── db_connection.py
└── server_health.py
```

and our first function will return something like:

```
    JSON

{
    "server": "LOCALHOST",
    "version": "...",
    "edition": "...",
    "database": "Vehicle"
}
```

Then we'll test it against your actual SQL Server.

This is the point where our project changes from "Python connected to SQL Server" into a real SQL Server monitoring project. 🚀

ok please go ahead on the server_health.py or what ever you want to start

Yes. Let's start with `server_health.py`, but we'll do it **properly and incrementally**.

Our first version will not try to collect CPU, memory, blocking, I/O, waits, etc. all at once. We'll first establish a clean monitoring pattern that we can reuse for every metric.

Stage 1 — Server Health

What we're building now

```
monitoring/
│
├── db_connection.py       ← connects to SQL Server
└── server_health.py       ← collects server information
```

The responsibility is deliberately separated:

`db_connection.py`

> How do I connect to SQL Server?

`server_health.py`

> What information can I collect from SQL Server?

This separation will become very useful later when the monitoring layer grows.

---

1. Create `server_health.py`

Inside:

```
D:\Python\AI_SQL_Monitor\monitoring
```

create:

server_health.py

Put this code in it:

Python Run

```python
from db_connection import get_connection


def get_server_health():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            SERVERPROPERTY('ServerName') AS server_name,
            SERVERPROPERTY('ProductVersion') AS product_version,
            SERVERPROPERTY('ProductLevel') AS product_level,
            SERVERPROPERTY('Edition') AS edition,
            DB_NAME() AS database_name;
        """

        cursor.execute(query)

        row = cursor.fetchone()

        return {
            "server": row.server_name,
            "product_version": row.product_version,
            "product_level": row.product_level,
            "edition": row.edition,
            "database": row.database_name
        }

    finally:
        connection.close()


if __name__ == "__main__":
    health = get_server_health()

    print(health)
```

---

2. Run it

You're already working from:

```
(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
```

So run:

PowerShell

```
python server_health.py
```

You should get something approximately like:

```
{
    'server': 'YOUR-PC\INSTANCE',
    'product_version': '16.x.x.x',
    'product_level': 'RTM',
    'edition': 'Developer Edition',
    'database': 'Vehicle'
}
```

Your values will obviously be different.

---

3. Let's understand what we just built

There are several important things here.

Python calls SQL Server

This:

Python Run

```
cursor.execute(query)
```

causes Python to send the SQL query to SQL Server.

The SQL itself is:

SQL

```
SELECT
    SERVERPROPERTY('ServerName'),
    SERVERPROPERTY('ProductVersion'),
    SERVERPROPERTY('ProductLevel'),
    SERVERPROPERTY('Edition'),
    DB_NAME();
```

Notice something important:

We're not querying your `Electric_Vehicle_Population_Data` table.

We're querying SQL Server's own system information.

That's the beginning of our monitoring system.

Why `SERVERPROPERTY()`?

SQL Server exposes server-level properties through:

```
SQL

SERVERPROPERTY(...)
```

For example:

```
SQL

SERVERPROPERTY('Edition')
```

might return:

```
Developer Edition
```

and:

```
SQL

SERVERPROPERTY('ProductVersion')
```

might return something like:

```
16.0.xxxx.x
```

So our Python application doesn't need to know beforehand which SQL Server version you're running.

It asks SQL Server.

---

4. Why return a dictionary?

This part is particularly important for our future AI architecture:

Python Run

```python
return {
    "server": row.server_name,
    "product_version": row.product_version,
    "product_level": row.product_level,
    "edition": row.edition,
    "database": row.database_name
}
```

We're deliberately converting the SQL result into **structured Python data**.

Eventually we'll have:

```
SQL Server
    ↓
Monitoring functions
    ↓
Structured data
    ↓
Rule Engine
    ↓
Agent
    ↓
LLM
```

For example, eventually:

JSON

```json
{
    "server": "LOCALHOST",
    "cpu_percent": 94,
    "memory_pressure": true,
    "active_sessions": 17,
    "blocking_sessions": 3,
    "long_running_queries": 2,
    "top_cpu_query": {...},
    "wait_statistics": {...}
}
```

That structured representation is going to become **the bridge between SQL Server monitoring and our AI layer**.

---

5. One design decision I want you to notice

We're doing:

Python Run

```python
connection = get_connection()
```

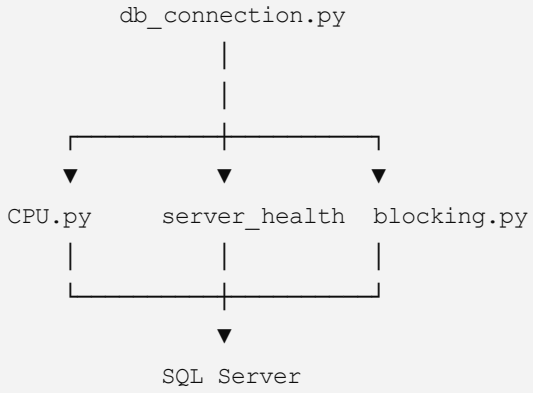
rather than putting the connection code directly inside `server_health.py`.

That's because later we'll have many monitoring functions:

```
get_server_health()
get_cpu_usage()
get_memory_usage()
get_active_sessions()
get_blocking_sessions()
get_long_running_queries()
get_top_cpu_queries()
get_top_io_queries()
get_wait_statistics()
get_database_status()
get_log_usage()
```

We don't want every function to contain its own connection logic.

Instead:



That's a small architectural decision now that will pay off later.

Your task now

Create `server_health.py` with the code above and run:

PowerShell

```
python server_health.py
```

Send me the output.

Then we'll inspect the result together and build the **first real performance metric: CPU usage**.

And that's where we'll start getting into the interesting SQL Server DMV work. 🚀

yes i do have knowledge on this. SO i understand what you aretyring to collect. usually this information it will collect from teh master database. with all the info

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe .\server_health.py
{'server': 'Moosa-DELLLaptop', 'product_version': '16.0.1000.6', 'product_level': 'RTM', 'edition': 'Developer Edition (64-bit)', 'database': 'Vehicle'}

Exactly. 👍 And your result is perfect.

```
server           = Moosa-DELLLaptop
product_version  = 16.0.1000.6
product_level    = RTM
edition          = Developer Edition (64-bit)
database         = Vehicle
```

And yes, your DBA understanding is correct: **for many server-level monitoring queries, we'll execute against** `master` because the relevant DMVs/system views expose instance-wide information there. We'll distinguish carefully between **server-level**, **database-level**, and **session/query-level** information as we build the collector.

One improvement before we move on

Our current `db_connection.py` hard-codes:

Python Run

```python
DATABASE = "Vehicle"
```

That's fine for proving connectivity, but for the monitoring agent I don't want us to tie the monitoring connection to the application/sample database.

Eventually we'll want:

```
SQL Server Instance
        |
        ├── master
        ├── Vehicle
        ├── tempdb
        ├── model
        └── ...
```

and our monitoring collector should be able to query the **instance**, regardless of which user database we're monitoring.

So let's make one small architectural improvement now.

Change `db_connection.py`

Change:

```python
DATABASE = "Vehicle"
```

to:

```python
DATABASE = "master"
```

Keep the rest unchanged.

Then run:

```powershell
python server_health.py
```

You'll probably now get:

```
'database': 'master'
```

That's actually what we want.

Then comes our first real monitoring metric: CPU

This will be more interesting because we'll move from:

> "Tell me information about the SQL Server."

to:

> **"Tell me what the SQL Server is doing right now."**

We'll investigate SQL Server's own performance information and eventually produce:

```
JSON

{
    "cpu_percent": 92,
    "timestamp": "...",
    "source": "SQL Server"
}
```

But before writing that, we'll discuss **exactly what CPU metric we're going to use and why**, because there are several ways to measure CPU and they don't all mean the same thing.

So make that small `master` change, run the script once, and send me the output. Then we'll start **CPU monitoring**.

how the db_conntion.py is called from server_health.py

how it was used here

Yes — this is an important Python concept, and in our project it is exactly how we're creating reusable modules.

Look at the first line of `server_health.py`:

```
Python                                                    Run

from db_connection import get_connection
```

That is how `db_connection.py` is being used.

Think of it like this

We have:

```
monitoring/
|
```

```
├── db_connection.py
```

Inside `db_connection.py`, we defined:

```python
def get_connection():
    ...
```

Then inside `server_health.py`, we say:

```python
from db_connection import get_connection
```

which means:

> **Go to `db_connection.py` and bring the `get_connection` function into this file.**

Now we can simply call:

```python
connection = get_connection()
```

The complete flow

When you run:

```powershell
python server_health.py
```

Python starts executing `server_health.py`.

It first encounters:

```python
from db_connection import get_connection
```

Python finds:

```
monitoring/db_connection.py
```

and loads the function:

Python Run

```python
def get_connection():
    ...
```

Now `server_health.py` has access to that function.

Then this happens:

Python Run

```python
def get_server_health():
    connection = get_connection()
```

So effectively:

```
server_health.py
       |
       | get_connection()
       ▼
db_connection.py
       |
       | pyodbc.connect(...)
       ▼
SQL Server
       |
       ▼
connection object
       |
       ▼
server_health.py
```

Let's make it very concrete

Your `db_connection.py` contains roughly:

Python Run

```python
import pyodbc

SERVER = "localhost"
DATABASE = "Vehicle"
DRIVER = "ODBC Driver 17 for SQL Server"


def get_connection():
    connection_string = (
```

```
        f"DRIVER={{{DRIVER}}};"
        f"SERVER={SERVER};"
        f"DATABASE={DATABASE};"
        "Trusted_Connection=yes;"
        "TrustServerCertificate=yes;"
    )

    return pyodbc.connect(connection_string)
```

The important part is:

Python Run

```
def get_connection():
```

That function **returns a connection object**.

Then `server_health.py` does:

Python Run

```
from db_connection import get_connection
```

and:

Python Run

```
connection = get_connection()
```

So `connection` now contains the connection returned by `pyodbc.connect()`.

Then:

Python Run

```
cursor = connection.cursor()
```

creates a cursor.

Then:

Python Run

```
cursor.execute(query)
```

sends the SQL query to SQL Server.

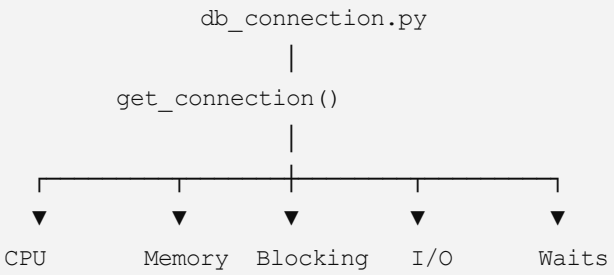
Why this design is useful

Imagine we eventually have 10 monitoring modules:

```
monitoring/
│
├── db_connection.py
├── server_health.py
├── cpu.py
├── memory.py
├── blocking.py
├── sessions.py
├── io.py
├── waits.py
├── queries.py
└── databases.py
```

We don't want this connection code copied into every file.

Instead:



Every module can reuse the same connection logic.

That's **modular design**.

And there's an even more important Python concept here

`db_connection.py` is a **module**.

A Python file can act as a module.

For example:

Python Run

```python
from db_connection import get_connection
```

means:

```
from <module> import <thing>
```

So:

```python
from db_connection import get_connection
```

means:

> From the `db_connection` module, import the `get_connection` function.

Similarly, later you might see:

```python
from cpu import get_cpu_usage
```

or:

```python
from blocking import get_blocking_sessions
```

This is how we'll gradually organize the monitoring system.

One subtle detail

Because both files are currently in the same directory:

```
monitoring/
├── db_connection.py
└── server_health.py
```

Python can find `db_connection` directly.

Later, when we turn `monitoring` into a proper Python package and run the application from the project root, we'll use package imports such as:

```python
from monitoring.db_connection import get_connection
```

We'll introduce that when it becomes useful. **No need to complicate it now.**

So your understanding should be:

> **`db_connection.py` owns the responsibility of creating the SQL Server connection. `server_health.py` imports and uses that connection to perform monitoring queries.**

That's exactly the separation we want in this project.

shall we move to the next step

Absolutely. 👍 **Let's move to the next step.**

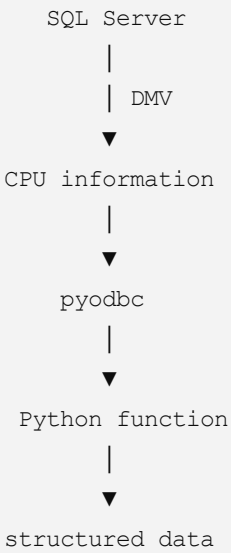
We have completed the first connectivity milestone, and you've also understood the module/import pattern. Now we can start collecting an **actual performance metric**.

Stage 1 — Step 2: CPU Monitoring

Our immediate objective is:

> **Get SQL Server CPU utilization from SQL Server itself using a DMV/system view, and return it as structured Python data.**

The architecture becomes:



First: an important DBA distinction

There are several different meanings of "CPU":

1. Windows/OS CPU

For example:

```
Overall server CPU = 92%
```

This tells us how busy the operating system's CPUs are.

2. SQL Server CPU

This tells us how much CPU SQL Server is consuming.

3. Query CPU

This tells us which individual queries are consuming CPU.

For our monitoring agent, **we eventually want all three perspectives**, because they answer different questions.

For example:

```
Windows CPU = 96%
SQL Server CPU = 94%
Top query CPU = 72%
```

That is much more useful than simply saying:

> CPU is high.

Our first CPU implementation

For the first version, we'll use SQL Server's performance counters exposed through:

```
sys.dm_os_performance_counters
```

This is a SQL Server DMV that exposes performance-counter information.

Let's first see what **your SQL Server actually returns** rather than hiding the SQL inside Python.

Step 1 — Run this in SSMS

Open a query window against your local SQL Server and run:

```sql
SELECT
    object_name,
    counter_name,
    instance_name,
    cntr_value
FROM sys.dm_os_performance_counters
```

```
WHERE counter_name LIKE '%CPU%'
ORDER BY counter_name;
```

Why do I want you to run it first?

Because we're building a monitoring system, not just copying a query.

I want you to see the raw information that SQL Server exposes.

You'll probably get several rows.

Step 2 — Find the SQL Server CPU counter

A commonly useful counter is:

```
SQLServer:Resource Pool Stats
CPU usage %
```

But the exact rows/counters exposed can vary depending on SQL Server configuration/version.

That's why we're going to inspect your actual output first.

Don't write `cpu.py` yet.

Run the query in SSMS and **paste the output here**.

Then we'll select the correct counter and build:

```
monitoring/
│
├── db_connection.py
├── server_health.py
└── cpu.py
```

with a function such as:

Python Run

```python
get_cpu_usage()
```

Eventually it will return something like:

JSON

```
{
    "metric": "cpu",
```

```
    "value": 23,
    "unit": "percent"
}
```

Then we'll take the next step and make it **time-aware**, because a single CPU reading isn't enough to implement your future rule:

```
CPU > 90% for 5 minutes
```

That distinction—**point-in-time metric vs monitoring over time**—will become very important when we build the Rule Engine.

i have created the cpu.py and copied the 35 line output of query to this cpu.py file. What next?

Yes — but **one important correction before we continue**.

If you literally copied the **35-line output** from SSMS into `cpu.py`, that's not what we want. The output is data; `cpu.py` should contain Python code that **executes the SQL query and converts the result into structured data**.

Don't worry—we can fix it easily.

1. Keep the SQL query we investigated

For now, let's use it inside Python.

Replace everything in `cpu.py` with:

Python Run

```python
from db_connection import get_connection


def get_cpu_counters():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            object_name,
            counter_name,
            instance_name,
```

```
            cntr_value
        FROM sys.dm_os_performance_counters
        WHERE counter_name LIKE '%CPU%'
        ORDER BY counter_name;
        """

        cursor.execute(query)

        rows = cursor.fetchall()

        results = []

        for row in rows:
            results.append({
                "object_name": row.object_name,
                "counter_name": row.counter_name,
                "instance_name": row.instance_name,
                "value": row.cntr_value
            })

        return results

    finally:
        connection.close()


if __name__ == "__main__":
    cpu_counters = get_cpu_counters()

    for counter in cpu_counters:
        print(counter)
```

2. Run it

You're already inside:

```
(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
```

Run:

PowerShell

```
python cpu.py
```

You should see Python dictionaries, something like:

```
{'object_name': '...', 'counter_name': '...', 'instance_name': '...', 'value': ...
{'object_name': '...', 'counter_name': '...', 'instance_name': '...', 'value': ...
...
```

What we've just learned

Notice the pattern:

```
SQL query
    ↓
cursor.fetchall()
    ↓
Python rows
    ↓
for row in rows
    ↓
Python dictionaries
    ↓
structured monitoring data
```

This is exactly the pattern we'll repeatedly use for our monitoring layer.

But don't worry about getting an actual `cpu_percent` yet. The query we're running now is deliberately returning the raw CPU-related counters so we can understand what SQL Server gives us.

Run:

PowerShell

```
python cpu.py
```

and **paste the output here**.

Then we'll identify the correct CPU counter from your actual SQL Server and turn it into a clean function:

Python Run

```
get_cpu_usage()
```

returning something like:

JSON

```
{
    "metric": "cpu",
    "value": 12.5,
    "unit": "percent"
}
```

That will be our first proper monitoring metric.

done, will proceed further

Great. 👍 Let's proceed.

Before we move to the next metric, I want to make one small change to our approach: **we'll finish CPU properly before moving to memory.** Your project instructions specifically call for reliable structured monitoring, so we shouldn't accumulate half-finished collectors.

Stage 1 — CPU Monitoring: Make It Useful

Right now `cpu.py` is successfully retrieving the raw CPU-related counters.

Our next goal is to transform that raw output into something the monitoring engine can actually use:

JSON

```
{
    "metric": "cpu",
    "value": 25.4,
    "unit": "percent"
}
```

But there is an important DBA concept here.

Why can't we simply use `cntr_value`?

Some SQL Server performance counters are **instantaneous values**, while others are **cumulative counters**.

For cumulative counters, one reading doesn't tell us utilization.

For example:

```
T1 → counter = 1000
T2 → counter = 1300
```

The useful information is the **change between T1 and T2**, not `1300` itself.

This is the same general principle you'll encounter later with:

- CPU
- I/O

- wait statistics
- query statistics

So our monitoring system needs to understand the nature of each metric.

Let's inspect your CPU output

Since you've already run:

```
PowerShell
python cpu.py
```

we can now identify which rows your SQL Server is exposing.

If you still have the output available, paste the output here.

If you don't, that's fine—just run:

```
PowerShell
python cpu.py
```

again and send me the output.

Then I'll show you exactly:

1. Which counter represents what.
2. Which one we should use.
3. Why we're selecting it.
4. How to calculate the actual CPU percentage.
5. How to turn it into `get_cpu_usage()`.

After that we'll have:

```
db_connection.py
        ↓
cpu.py
        ↓
get_cpu_usage()
        ↓
{
    "metric": "cpu",
    "value": ...,
    "unit": "percent"
}
```

Then—and only then—we'll move to memory monitoring.

Pasted text.txt
Document

Here is the output of the cpy.py

Perfect. This output is exactly what we needed. Pasted text

You have the important counter:

```
SQLServer:Resource Pool Stats
CPU usage %
instance = default
value = 0
```

and:

```
instance = internal
value = 5
```

There is also:

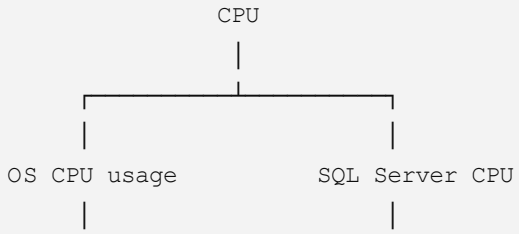
```
SQLServer:Workload Group Stats
CPU usage %
```

But here's an important DBA point

We **shouldn't simply take** `internal = 5` **and call it "SQL Server CPU = 5%"**.

`CPU usage %` under Resource Pool/Workload Group Stats represents CPU usage for those particular resource pools/workload groups. It isn't necessarily the same thing as **overall Windows CPU utilization**.

For our monitoring agent, I want to separate these concepts properly:



```
      Windows metrics       SQL Server DMVs
            |                     |
            └──────────┬──────────┘
                       |
                  Monitoring
```

And later:

```
SQL Server CPU
      |
      ├── Which queries?
      |
      ├── How much CPU?
      |
      ├── What waits?
      |
      └── Is CPU actually the bottleneck?
```

That's much more useful for a DBA/AI agent.

Let's build `get_cpu_usage()`

For our first SQL Server CPU metric, we'll use the SQL Server performance counter you just found.

Replace the contents of `cpu.py` with:

Python Run

```python
from db_connection import get_connection


def get_cpu_usage():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            instance_name,
            cntr_value
        FROM sys.dm_os_performance_counters
        WHERE object_name LIKE '%Resource Pool Stats%'
          AND counter_name = 'CPU usage %'
          AND instance_name = 'default';
        """

        cursor.execute(query)
```

```
        row = cursor.fetchone()

        if row is None:
            return {
                "metric": "sql_server_cpu",
                "value": None,
                "unit": "percent",
                "status": "not_available"
            }

        return {
            "metric": "sql_server_cpu",
            "value": row.cntr_value,
            "unit": "percent",
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":
    cpu = get_cpu_usage()
    print(cpu)
```

Run it

From:

```
(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
```

run:

```
PowerShell

python cpu.py
```

Based on your current output, we expect something like:

```
{'metric': 'sql_server_cpu', 'value': 0, 'unit': 'percent', 'status': 'ok'}
```

That's perfectly fine.

Your laptop is probably not doing anything CPU-intensive against SQL Server at the moment.

But don't be surprised by `0`

This is actually an important lesson.

If SQL Server is basically idle:

```
SQL Server CPU = 0%
```

is entirely reasonable.

Later we'll deliberately create CPU load and see whether our collector responds.

That's part of your Stage 2:

> **Create controlled problems.**

We'll eventually make SQL Server work hard and observe:

```
CPU
  5%
  ↓
  20%
  ↓
  60%
  ↓
  90%+
```

Then our monitoring system can detect:

```
CPU > 90%
      ↓
WARNING
      ↓
AI investigation
```

One architectural improvement we'll make later

For now:

Python Run

```python
get_cpu_usage()
```

returns the **current SQL Server Resource Pool CPU counter**.

Later, I want our monitoring layer to return a richer structure:

```json
{
    "timestamp": "...",
    "server": "Moosa-DELLLaptop",
    "metrics": {
        "sql_server_cpu_percent": 5,
        "os_cpu_percent": 12,
        "active_sessions": 4,
        "blocking_sessions": 0
    }
}
```

And eventually the Rule Engine will operate on that structure.

But **we're not building that yet**.

Your next action

Run:

```powershell
python cpu.py
```

and send me the result.

Once we confirm `get_cpu_usage()` works, we'll move to **Memory Pressure**, where we'll start using another important SQL Server DMV and you'll see why SQL Server memory monitoring is different from simply checking Windows RAM usage.

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe .\cpu.py
{'metric': 'sql_server_cpu', 'value': 0, 'unit': 'percent', 'status': 'ok'}

will go next

Perfect. ✅ **CPU monitoring is working.**

We now have our first genuine monitoring function:

```
get_cpu_usage()
       ↓
SQL Server DMV
       ↓
Python
       ↓
{
    "metric": "sql_server_cpu",
    "value": 0,
    "unit": "percent",
    "status": "ok"
}
```

Let's move to the next metric.

Stage 1 — Step 3: Memory Monitoring

This one is particularly important for a DBA because we don't want to make the mistake of treating:

> **Windows free memory = SQL Server memory pressure**

They are related, but they aren't the same thing.

Our monitoring eventually needs to distinguish:

```
Windows / OS memory
        │
        └── Is the server under OS memory pressure?

SQL Server memory
        │
        └── Is SQL Server under memory pressure?

SQL Server configuration
        │
        └── Is max server memory configured sensibly?
```

Our first target

We'll collect SQL Server's memory state using:

```
    SQL

sys.dm_os_sys_memory
```

This DMV gives us information about the operating system's memory as seen by SQL Server.

We'll also eventually use:

```
SQL

sys.dm_os_process_memory
```

to understand SQL Server's process-level memory.

And later:

```
SQL

sys.dm_os_sys_info
```

for system-level information.

Step 1 — Let's inspect the DMV first

Before creating `memory.py`, run this in SSMS:

```
SQL

SELECT
    total_physical_memory_kb,
    available_physical_memory_kb,
    total_page_file_kb,
    available_page_file_kb,
    system_memory_state_desc
FROM sys.dm_os_sys_memory;
```

You should get **one row**.

The important column for us initially is:

```
system_memory_state_desc
```

It may say something like:

```
Available physical memory is high
```

or another state depending on your system.

Why start with this?

Because our agent eventually needs to reason from **evidence**, not just produce a number.

For example:

```
Available physical memory = 1.2 GB
System memory state = Available physical memory is low
```

is much more meaningful than:

```
Memory = 95%
```

Later we can combine:

```
OS memory
+
SQL Server memory usage
+
PLE
+
Memory Grants Pending
+
Page Life Expectancy
+
Wait statistics
```

to determine whether we're actually dealing with SQL Server memory pressure.

Don't create `memory.py` yet.

Run the query in SSMS and paste the result here.

We'll then build `memory.py` based on **your actual SQL Server output**, just as we did with CPU.

33292992 16757412 38273728 18789740 Available physical memory is high
i ahve 32 gb i was using only 16 Rest 18 is avialable

we can create the memory.py now

Exactly. 👍 And your interpretation is basically right.

Your result is:

```
total physical memory       33,292,992 KB
available physical memory   16,757,412 KB
system memory state         Available physical memory is high
```

33,292,992 KB is approximately **31.75 GiB**, so your "32 GB" laptop is consistent with what SQL Server sees.

And yes, roughly half is currently available. The important thing is that SQL Server itself reports:

> **Available physical memory is high**

We'll let SQL Server tell us that rather than making our own arbitrary assumption.

Create `memory.py`

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\memory.py
```

Use this:

Python Run

```python
from db_connection import get_connection


def get_memory_status():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            total_physical_memory_kb,
            available_physical_memory_kb,
            total_page_file_kb,
            available_page_file_kb,
            system_memory_state_desc
        FROM sys.dm_os_sys_memory;
        """

        cursor.execute(query)
```

```
        row = cursor.fetchone()

        if row is None:
            return {
                "metric": "memory",
                "status": "not_available"
            }

        total_memory_gb = row.total_physical_memory_kb / 1024 / 1024
        available_memory_gb = row.available_physical_memory_kb / 1024 / 1024
        used_memory_gb = total_memory_gb - available_memory_gb

        return {
            "metric": "memory",
            "total_physical_memory_gb": round(total_memory_gb, 2),
            "available_physical_memory_gb": round(available_memory_gb, 2),
            "used_physical_memory_gb": round(used_memory_gb, 2),
            "memory_state": row.system_memory_state_desc,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":
    memory = get_memory_status()
    print(memory)
```

Run it

From your `monitoring` directory:

PowerShell

```
python memory.py
```

You should get something similar to:

```
{
    'metric': 'memory',
    'total_physical_memory_gb': 31.75,
    'available_physical_memory_gb': 15.98,
    'used_physical_memory_gb': 15.77,
    'memory_state': 'Available physical memory is high',
    'status': 'ok'
}
```

Your exact numbers will be slightly different.

Notice what we're doing here

We received:

```
33292992 KB
16757412 KB
```

but our monitoring application converts them into:

```
31.75 GB
15.98 GB
```

That's much more useful for the eventual dashboard and AI agent.

The conversion is:

Python Run

```python
kb / 1024 / 1024
```

because:

```
KB → MB → GB
```

One important improvement

You may notice that we're returning:

```
used_physical_memory_gb
```

This is calculated as:

```
total - available
```

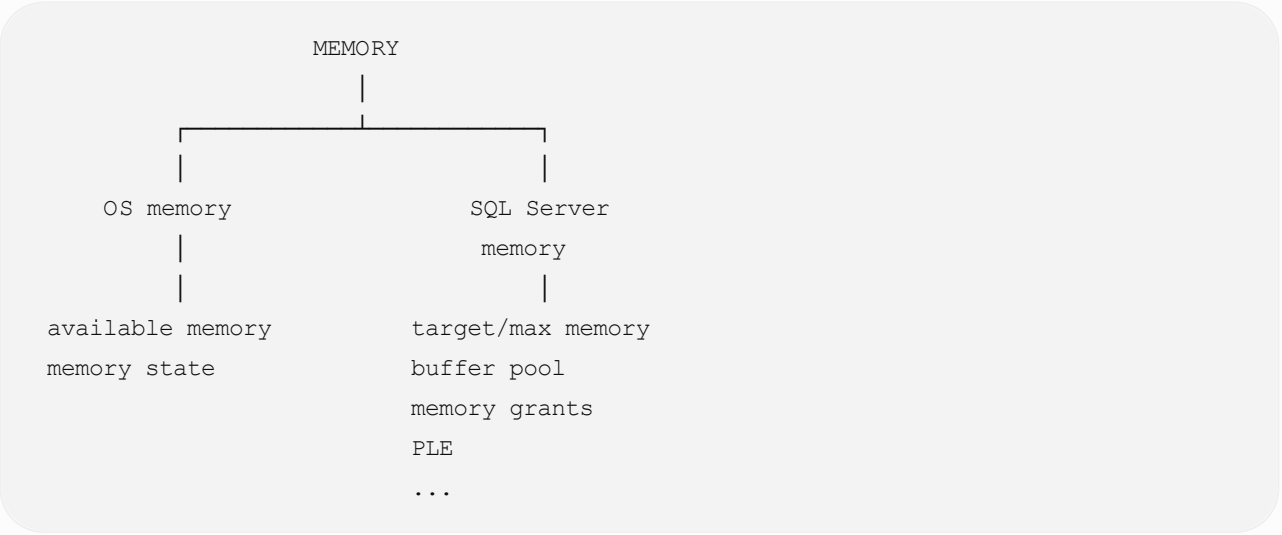
That's useful for displaying information, but **we should not yet call this "SQL Server memory used."**

It's actually:

> **Physical memory used from the OS perspective.**

That's an important distinction for our future AI agent.

Eventually we'll collect something more like:



Then the agent can reason:

> "The OS has sufficient memory, but SQL Server is experiencing memory pressure."

or:

> "The entire machine is under memory pressure."

Those are **very different DBA diagnoses**.

Your task

Create `memory.py`, run:

PowerShell

```
python memory.py
```

and send me the output.

Then we'll move to **active sessions and blocking**, which is where our monitoring system starts becoming much more DBA-oriented.

Got it here i shte output, shall we proceed next

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe .\memory.py
{'metric': 'memory', 'total_physical_memory_gb': 31.75, 'available_physical_memory_gb': 16.7, 'used_physical_memory_gb': 15.05, 'memory_state': 'Available physical memory is high', 'status': 'ok'}

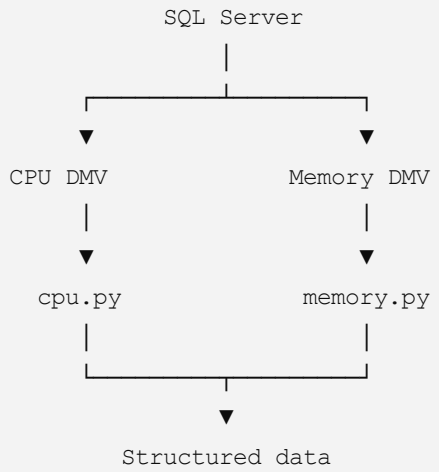
Yes. ✅ **Memory monitoring is working correctly**, and the output makes sense:

```
Total memory      : 31.75 GB
Available memory  : 16.70 GB
Used memory       : 15.05 GB
Memory state      : Available physical memory is high
Status            : OK
```

So we now have two working monitoring components:

```
monitoring/
│
├── db_connection.py
├── server_health.py
├── cpu.py
└── memory.py
```

And the flow is becoming:



Next: Active Sessions

Now we'll start getting into **actual DBA monitoring**.

Our next question is:

> **Who is currently connected to SQL Server, and what are those sessions doing?**

We'll use:

```
SQL

sys.dm_exec_sessions
```

This DMV gives us session-level information such as:

- session ID
- login name
- host name
- program name
- status
- database
- login time

Then we'll combine it with:

```
SQL

sys.dm_exec_requests
```

which tells us about **currently executing requests**.

This distinction is important:

```
Session
   =
Connection to SQL Server

Request
   =
Work currently being executed by that session
```

For example:

```
Session 52
   |
   ├── Login: Moosa
   ├── Host: Laptop
   └── Request
         |
         └── SELECT ... currently running
```

This becomes the foundation for our later:

- blocking detection
- long-running query detection
- top CPU queries
- top I/O queries

Step 1 — Inspect sessions in SSMS

Run this first:

```sql
SELECT
    session_id,
    login_name,
    host_name,
    program_name,
    status,
    database_id,
    login_time
FROM sys.dm_exec_sessions
WHERE is_user_process = 1
ORDER BY session_id;
```

Why `is_user_process = 1`?

Because SQL Server itself has internal/system sessions.

We don't want our initial monitoring output polluted with those.

So we're saying:

> Show me sessions belonging to users/applications.

What we'll build next

After seeing your output, we'll create:

```
monitoring/
|
├── db_connection.py
├── server_health.py
├── cpu.py
```

```
├── memory.py
```

and eventually:

```python
get_active_sessions()
```

returning something like:

```json
{
    "metric": "active_sessions",
    "count": 4,
    "sessions": [
        {
            "session_id": 52,
            "login_name": "Moosa",
            "host_name": "Moosa-DELLLaptop",
            "status": "running"
        }
    ],
    "status": "ok"
}
```

Run the SSMS query and send me the output. Then we'll build `sessions.py`.

SELECT session_id, login_name, host_name, program_name, status, database_id, login_time FROM sys.dm_exec_sessions WHERE is_user_process = 1 ORDER BY session_id;

Yes — that is the **query itself**, but I need the **result rows** from SSMS, not the SQL statement.

After you execute it in SSMS, you should see a grid underneath, something like:

```
session_id | login_name | host_name | program_name | status | database_id | login_
-------------------------------------------------------------------------------------
52         | Moosa      | ...       | SSMS         | running| 7           | ...
53         | ...        | ...       | ...          | sleeping| 7           | ...
```

Please **copy the rows from the Results grid** and paste them here.

If there are many rows, that's completely fine—send all of them.

Why I want the actual result

We're going to use your real output to understand the difference between:

```
Session
   ↓
Connection
   ↓
Request
   ↓
Currently executing work
```

Then we'll create `sessions.py` and turn those rows into structured monitoring data.

So for now, **don't create/change `sessions.py` yet**.

Just execute the query and paste the **result table** here.

session_id login_name host_name program_name status database_id login_time
51 NT SERVICE\SQLTELEMETRY MOOSA-DELLLAPTO SQLServerCEIP sleeping 1 2026-09-09 12:13:29.353
52 MOOSA-DELLLAPTO\Moosa MOOSA-DELLLAPTO Microsoft SQL Server Management Studio - Query sleeping 1 2026-09-09 11:55:42.580
61 MOOSA-DELLLAPTO\Moosa MOOSA-DELLLAPTO Microsoft SQL Server Management Studio - Query running 1 2026-09-09 12:16:35.910

Show more

Perfect. Now we have the actual session data, and this is a good point to build `sessions.py`.

One useful observation from your output: `sleeping` **does not mean a problem**. It generally means the session is connected but isn't currently executing a request. Your session `61` is the one currently shown as `running`.

Step 1 — Create `sessions.py`

Create it under:

```
D:\Python\AI_SQL_Monitor\monitoring\sessions.py
```

Use:

Python Run

```python
from db_connection import get_connection


def get_user_sessions():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            session_id,
            login_name,
            host_name,
            program_name,
            status,
            database_id,
            login_time
        FROM sys.dm_exec_sessions
        WHERE is_user_process = 1
        ORDER BY session_id;
        """

        cursor.execute(query)

        rows = cursor.fetchall()

        sessions = []

        for row in rows:
            sessions.append({
                "session_id": row.session_id,
                "login_name": row.login_name,
                "host_name": row.host_name,
                "program_name": row.program_name,
                "status": row.status,
                "database_id": row.database_id,
                "login_time": str(row.login_time)
            })

        return {
            "metric": "user_sessions",
            "count": len(sessions),
```

```
            "sessions": sessions,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":
    result = get_user_sessions()

    print(f"User session count: {result['count']}")

    for session in result["sessions"]:
        print(session)
```

Step 2 — Run it

PowerShell

```
python sessions.py
```

You should get something similar to:

```
User session count: 6

{'session_id': 51, ...}
{'session_id': 52, ...}
...
```

The exact number may change because SSMS itself creates sessions while you're working.

But notice something important

We're deliberately **not calling all sessions "active sessions."**

Our function is:

Python Run

```
get_user_sessions()
```

because your query returns:

```
sleeping
running
```

A better DBA model is:

```
User Sessions
      |
      ├── Running
      |
      └── Sleeping
```

Later we'll calculate:

```
total_user_sessions
running_sessions
sleeping_sessions
```

And then we'll introduce **requests**.

That's where things become much more interesting:

```
Session 61
    |
    └── Request
          |
          ├── SQL text
          ├── CPU time
          ├── elapsed time
          ├── logical reads
          ├── wait type
          └── blocking session
```

That information is what eventually allows our AI agent to answer something like:

> "Why is SQL Server slow right now?"

rather than simply:

> "There are six sessions."

One thing I want you to notice in your current result

You have:

```
51  SQLTELEMETRY  sleeping
52  Moosa         sleeping
```

```
61  Moosa          running
62  Moosa          sleeping
63  Moosa          sleeping
81  Moosa          sleeping
```

So at that moment:

6 user sessions

and:

1 running session

That's exactly the sort of information our monitoring layer should capture.

Your turn

Create `sessions.py` and run:

PowerShell

```
python sessions.py
```

Send me the output.

Then we'll build the **request/query monitor**. That's the step where we start connecting sessions to actual SQL workload, and it will also prepare us for **blocking and long-running query detection**.

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python sessions.py
User session count: 6
{'session_id': 51, 'login_name': 'NT SERVICE\\SQLTELEMETRY', 'host_name': 'MOOSA-DELLLAPTO', 'program_name': 'SQLServerCEIP', 'status': 'sleeping', 'database_id': 1, 'login_time': '2026-09-09 12:18:30.677000'}
{'session_id': 52, 'login_name': 'MOOSA-DELLLAPTO\\Moosa', 'host_name': 'MOOSA-DELLLAPTO', 'program_name': 'Microsoft SQL Server

Show more

Perfect. ✅ This is working exactly as expected.

And there's an interesting change from your previous result:

```
Session 63
Program: Python
Status: running
```

That's actually **our monitoring script itself**. While `sessions.py` was querying SQL Server, SQL Server saw the Python connection as a running user session. Once the query finished, it became sleeping.

That's a nice real-world demonstration of the difference between **session** and **request**.

Next Step — Query / Request Monitoring

Now we move one level deeper.

We currently know:

```
Session 63
   ↓
Python connection
   ↓
running
```

But we don't yet know:

> **What SQL is that session executing?**

For that, we'll use:

```
SQL

sys.dm_exec_requests
```

and combine it with:

```
SQL

sys.dm_exec_sql_text()
```

This is where our monitoring starts becoming genuinely useful.

The relationship

Think of SQL Server like this:

```
SESSION
  │
  │  "I am connected"
  │
  ▼
REQUEST
  │
  │  "I am currently doing this"
  │
  ├── SQL text
  ├── CPU time
  ├── elapsed time
  ├── reads
  ├── writes
  ├── wait type
  └── blocking session
```

A sleeping session generally has **no currently executing request**.

A running session may have a request.

Step 1 — Test the DMV in SSMS

Run this:

SQL

```sql
SELECT
    r.session_id,
    r.status,
    r.command,
    r.cpu_time,
    r.total_elapsed_time,
    r.reads,
    r.writes,
    r.logical_reads,
    r.wait_type,
    r.wait_time,
    r.blocking_session_id,
    DB_NAME(r.database_id) AS database_name,
    s.login_name,
    s.host_name,
    s.program_name,
    t.text AS sql_text
FROM sys.dm_exec_requests r
INNER JOIN sys.dm_exec_sessions s
    ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
```

```
WHERE s.is_user_process = 1
ORDER BY r.cpu_time DESC;
```

What we're collecting

Column	Why we care
`session_id`	Identify the session
`status`	Running / suspended etc.
`command`	What SQL Server is executing
`cpu_time`	CPU consumed by request
`total_elapsed_time`	How long it's been running
`reads`	Physical reads
`writes`	Writes
`logical_reads`	Buffer/cache activity
`wait_type`	What the request is waiting for
`wait_time`	How long it's waiting
`blocking_session_id`	Who is blocking it
`database_name`	Database involved
`sql_text`	Actual SQL being executed

This single query is going to become **one of the most important foundations of our monitoring agent**.

One thing you may notice

When you execute the query manually in SSMS, the result might be:

```
0 rows
```

That's completely possible.

Why?

Because `sys.dm_exec_requests` shows **currently executing requests**.

By the time you run the query, your previous Python request may already have finished.

That's actually why later our monitoring agent will need to **poll continuously**, rather than execute once.

Eventually:

```
Every 5 seconds
       ↓
Collect metrics
       ↓
Store snapshot
       ↓
Compare with thresholds
       ↓
Detect anomaly
       ↓
Investigate
```

But we're not building the scheduler yet.

For now

Run the SQL query above in SSMS and send me the result.

If it returns **0 rows**, tell me that too. Don't worry—that's a valid result.

Then we'll create:

```
monitoring/
├── db_connection.py
├── server_health.py
├── cpu.py
├── memory.py
├── sessions.py
└── requests.py
```

and build `get_active_requests()`.

After that, we'll tackle **blocking**, which is where we can deliberately create a blocking scenario in your `Vehicle` database and watch our monitoring system detect it.

```
SELECT
```

```
    r.session_id,
    r.status,
    r.command,
    r.cpu_time,
    r.total_elapsed_time
```

Show more

Yes — I see the query. But again, what you pasted is the **SQL statement**, not the **result from the Results grid**.

Please execute it in SSMS by pressing **F5** / clicking **Execute**.

Then look underneath the query at the **Results** tab.

You may get something like:

```
session_id | status | command | cpu_time | elapsed | ... | sql_text
```

or you may get:

```
(0 rows affected)
```

If you get 0 rows

That's perfectly fine. In fact, it's likely because there is no user request actively executing at the exact moment you run the query.

If you want to make the result easier to capture

Run this slightly simpler version first:

SQL

```sql
SELECT
    r.session_id,
    r.status,
    r.command,
    r.cpu_time,
    r.total_elapsed_time,
    r.reads,
    r.writes,
    r.logical_reads,
    r.wait_type,
```

```
    r.wait_time,
    r.blocking_session_id,
    DB_NAME(r.database_id) AS database_name,
    s.login_name,
    s.host_name,
    s.program_name
FROM sys.dm_exec_requests r
INNER JOIN sys.dm_exec_sessions s
    ON r.session_id = s.session_id
WHERE s.is_user_process = 1
ORDER BY r.cpu_time DESC;
```

Please paste the rows that appear under Results, not the query text.

Once I see that, we'll create `requests.py`.

session_id status command cpu_time total_elapsed_time reads writes logical_reads wait_type wait_time blocking_session_id database_name login_name host_name program_name sql_text
61 running SELECT 0 0 0 0 0 NULL 0 0 master MOOSA-DELLLAPTO\Moosa MOOSA-DELLLAPTO Microsoft SQL Server Management Studio - Query SELECT r.session_id, r.status, r.command, r.cpu_time, r.total_elapsed_time, r.reads, r.writes, r.logical_reads, r.wait_type, r.wait_time,

Show more

Perfect. Now we have the **actual request output**, and it shows exactly what we expected.

Your request was:

```
session_id      61
status          running
command         SELECT
cpu_time        0
elapsed_time    0
blocking        0
database        master
program         SSMS
```

And notice the `sql_text` — it is **the monitoring query itself**. That's because while SSMS is executing the query, SQL Server exposes that query through `sys.dm_exec_requests`.

This is an important concept for our project:

> **Our monitoring query can see itself while it is running.**

Step 4 — Create `requests.py`

Now let's turn this into a reusable Python monitoring function.

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\requests.py
```

Put this in it:

Python Run

```python
from db_connection import get_connection


def get_active_requests():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            r.session_id,
            r.status,
            r.command,
            r.cpu_time,
            r.total_elapsed_time,
            r.reads,
            r.writes,
            r.logical_reads,
            r.wait_type,
            r.wait_time,
            r.blocking_session_id,
            DB_NAME(r.database_id) AS database_name,
            s.login_name,
            s.host_name,
            s.program_name,
            t.text AS sql_text
        FROM sys.dm_exec_requests r
        INNER JOIN sys.dm_exec_sessions s
            ON r.session_id = s.session_id
        CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
```

```
        WHERE s.is_user_process = 1
        ORDER BY r.cpu_time DESC;
        """

        cursor.execute(query)

        rows = cursor.fetchall()

        requests = []

        for row in rows:
            requests.append({
                "session_id": row.session_id,
                "status": row.status,
                "command": row.command,
                "cpu_time_ms": row.cpu_time,
                "elapsed_time_ms": row.total_elapsed_time,
                "reads": row.reads,
                "writes": row.writes,
                "logical_reads": row.logical_reads,
                "wait_type": row.wait_type,
                "wait_time_ms": row.wait_time,
                "blocking_session_id": row.blocking_session_id,
                "database_name": row.database_name,
                "login_name": row.login_name,
                "host_name": row.host_name,
                "program_name": row.program_name,
                "sql_text": row.sql_text
            })

        return {
            "metric": "active_requests",
            "count": len(requests),
            "requests": requests,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":
    result = get_active_requests()

    print(f"Active request count: {result['count']}")

    for request in result["requests"]:
        print(request)
```

Step 5 — Run it

From:

```
(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
```

run:

```
PowerShell
python requests.py
```

You might get:

```
Active request count: 1
```

or:

```
Active request count: 0
```

Both can be valid.

Why?

Remember:

```
sys.dm_exec_requests
```

shows **currently executing requests**.

Our Python script itself creates a request while it is executing the query, so you may see the monitoring query itself.

Once the query finishes, that request disappears.

We now have an important foundation

Our monitoring directory is becoming:

```
monitoring/
│
├── db_connection.py
│
├── server_health.py
│
├── cpu.py
│
```

```
├── memory.py
│
├── sessions.py
│
└── requests.py
```

And we're collecting:

```
Server
   │
   ├── Server information
   ├── CPU
   ├── Memory
   ├── Sessions
   └── Active Requests
```

But **don't add more metrics yet**.

The next thing I want us to do is much more interesting:

🔥 Blocking Detection

We'll use:

```
SQL

blocking_session_id
```

which you already saw in the request output.

Ultimately our monitoring system should be able to say:

```
🚨 BLOCKING DETECTED

Victim Session : 63
Blocking Session: 52
Database       : Vehicle
Wait Type      : LCK_M_X
Wait Time      : 18,420 ms
```

And then later our AI layer can investigate:

> Who is blocking whom, what SQL caused it, how long has it been blocked, and what should the DBA do?

That is much closer to the actual goal of your AI SQL Monitoring project.

Run `requests.py` now and send me its output.

venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe .\requests.py
Active request count: 1
{'session_id': 52, 'status': 'running', 'command': 'SELECT', 'cpu_time_ms': 4, 'elapsed_time_ms': 8, 'reads': 80, 'writes': 0, 'logical_reads': 1271, 'wait_type': None, 'wait_time_ms': 0, 'blocking_session_id': 0, 'database_name': 'master', 'login_name': 'MOOSA-DELLLAPTO\\Moosa', 'host_name': 'MOOSA-DELLLAPTO', 'program_name': 'Python', 'sql_text': '\n SELECT\n r.session_id,\n r.status,\n r.command,\n r.cpu_time,\n

Show more

Excellent. ✅ `requests.py` is working correctly.

And this result is actually useful because we can see the monitoring query **observing itself**:

```
session_id          52
status              running
command             SELECT
cpu_time            4 ms
elapsed_time        8 ms
reads               80
logical_reads       1271
blocking_session    0
database            master
program             Python
```

Most importantly:

```
blocking_session_id = 0
```

means **this request is not currently blocked**.

One thing we should fix now

Our current query includes the monitoring application's own request.

That's not desirable for the final monitoring system.

Otherwise our agent could eventually say:

> "The top CPU query is my own monitoring query." 😄

We should exclude our monitoring connection from the workload analysis.

There are several ways to do that. For now, the cleanest approach is to exclude:

```
SQL

program_name = 'Python'
```

from the request query.

Change this:

```
SQL

WHERE s.is_user_process = 1
```

to:

```
SQL

WHERE s.is_user_process = 1
  AND s.program_name <> 'Python'
```

Then run:

```
PowerShell

python requests.py
```

You may now get:

```
Active request count: 0
```

That's perfectly normal.

🔥 Now we're ready for Blocking

This is where the project becomes much more interesting.

We don't want to simply write a query and hope blocking happens. **We're going to deliberately create a blocking scenario in your `Vehicle` database**, then have our

monitoring code detect it.

We'll create two SQL sessions:

```
Session A
   │
   │ BEGIN TRANSACTION
   │ UPDATE ...
   │
   ▼
Locks a row
```

Then:

```
Session B
   │
   │ UPDATE same row
   ▼
WAITING
   │
   └── blocked by Session A
```

Our monitoring agent should eventually discover:

```
Victim session       : B
Blocking session     : A
Wait type            : LCK_M_...
Blocking session ID  : A
```

And then we can retrieve:

```
Blocking SQL
Victim SQL
Database
Wait duration
Login
Host
```

That's the foundation for an AI diagnosis such as:

> **Blocking detected. Session 52 is holding a transaction open and preventing Session 63 from acquiring the required lock.**

Before we create the blocking test

We need to be careful with your sample database.

We're going to use a **test transaction and roll it back**, so we don't permanently modify your `Electric_Vehicle_Population_Data`.

I want to do this step-by-step because there are two SSMS windows involved.

First, make the small `requests.py` change

Use:

```sql
WHERE s.is_user_process = 1
  AND s.program_name <> 'Python'
```

Run the script once.

Then tell me the result.

After that I'll give you **Session A / Session B blocking test instructions**, and we'll watch `requests.py` detect the blocking in real time.

(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
python.exe .\requests.py
Active request count: 0

please go ahead with blocking

Perfect. `requests.py` is now clean — it doesn't report its own Python monitoring query. ✅

Now let's deliberately create **blocking**.

We'll do this safely using your `Vehicle` database and `ROLLBACK`.

Step 1 — Open two SSMS query windows

Open **two separate query windows**, both connected to:

```
Server: localhost
Database: Vehicle
```

Call them mentally:

```
Window A = Blocking session
Window B = Victim session
```

Step 2 — Window A: create the lock

First, let's identify a row we can safely use.

Run in **Window A**:

```
SQL

USE Vehicle;
GO

SELECT TOP 1 *
FROM Electric_Vehicle_Population_Data;
```

You need to know the table's primary key/unique column so that we can safely target one row.

Important

I don't want to guess the column name.

Run:

```
SQL

EXEC sp_help 'Electric_Vehicle_Population_Data';
```

Look for the **Primary Key** column.

For example, it might be something like:

```
VIN
```

or another column.

Don't execute the UPDATE yet.

Why we're doing this carefully

The blocking demonstration needs two sessions to touch the **same locked resource**:

```
Window A
   │
   │ BEGIN TRAN
   │ UPDATE row X
   │
   ▼
🔒 Row X locked
   │
   │
   │
Window B
   │
   │ UPDATE row X
   ▼
⏳ WAITING
```

Then our Python monitor will see:

```
blocking_session_id = <Window A session ID>
```

We'll also retrieve the SQL text of both sessions.

Your next action

In **Window A**, run:

```
SQL

EXEC sp_help 'Electric_Vehicle_Population_Data';
```

and paste the part showing the **column names and primary key**.

Once I know the key column, I'll give you the exact safe `BEGIN TRAN / UPDATE` statement for Window A and the matching blocking statement for Window B.

Don't run any UPDATE yet.

Pasted text(1).txt
Document

Here is the sp_help of the table in Text format

Perfect. We have the table structure now. The important finding is that **there is no primary key and no index** on `Electric_Vehicle_Population_Data`. Pasted text

That's actually fine for our blocking lab. We can use the `VIN (1-10)` column to identify a row, but because there is no index, I want to make the test predictable by using a **table-level lock** rather than trying to demonstrate a row lock.

🔒 Blocking Lab — Safe Test

We'll use two SSMS windows.

Window A — Blocking session

Make sure the database is `Vehicle`, then run:

SQL

```sql
USE Vehicle;
GO

BEGIN TRANSACTION;

SELECT TOP 1 *
FROM dbo.Electric_Vehicle_Population_Data
WITH (TABLOCKX);

-- DO NOT COMMIT OR ROLLBACK YET
```

The important part is:

SQL

```sql
WITH (TABLOCKX)
```

This requests an **exclusive table lock**.

Keep this transaction open.

Do not close Window A.

Step 2 — Window B — Victim session

Open another SSMS query window and run:

SQL

```
USE Vehicle;
GO

SELECT TOP 1 *
FROM dbo.Electric_Vehicle_Population_Data;
```

This query should **hang/wait**.

That's exactly what we want.

Your situation should now look conceptually like:

```
Window A
Session 100
    |
    ├── BEGIN TRANSACTION
    |
    ├── TABLOCKX
    |
    └── 🔒 Vehicle table
            |
            |
            ▼
       Window B
       Session 101
            |
            └── SELECT
                  |
                  └── ⏳ WAITING
```

⚠️ Don't wait too long

Once Window B is visibly waiting, **leave both windows open**.

Now open your PowerShell and run:

PowerShell

```
python requests.py
```

This is the interesting part.

Our Python monitor should see something similar to:

```
blocking_session_id: 100
wait_type: LCK_M_S
wait_time_ms: ...
```

The exact session IDs and wait type will depend on what SQL Server reports.

What we're proving

We're going to connect three things:

```
Session
   ↓
Request
   ↓
Blocking
```

Previously we collected:

```
session_id
login_name
host_name
program_name
```

and then:

```
cpu_time
elapsed_time
reads
writes
wait_type
blocking_session_id
sql_text
```

Now we're deliberately creating:

```
blocking_session_id != 0
```

That gives our future Rule Engine something meaningful to detect:

```
IF blocking_session_id > 0
THEN blocking_detected = true
```

But **don't build the rule yet**. First let's observe the raw SQL Server behavior.

Do this now

1. Window A → execute the `BEGIN TRANSACTION` + `TABLOCKX`.
2. Window B → execute the `SELECT`.
3. Confirm Window B is waiting.
4. Run:

PowerShell

```
python requests.py
```

5. **Paste the output here.**

Once we see the blocking information, I'll explain exactly what SQL Server is telling us and then we'll create `blocking.py`.

as a dba i know how to make this block with the begin tran. I am just simple followign your step.

here is the output of hte pythong file

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe .\requests.py
Active request count: 1
{'session_id': 73, 'status': 'suspended', 'command': 'SELECT', 'cpu_time_ms': 0, 'elapsed_time_ms': 58995, 'reads': 1, 'writes': 0, 'logical_reads': 14, 'wait_type':

Show more

Exactly. 👍 And your result is **excellent for our project**. You followed the test correctly, and we now have a real blocking event captured by our Python monitor.

The key part is:

```
session_id           73
status               suspended
wait_type            LCK_M_IS
wait_time_ms         58993
blocking_session_id  71
database_name        Vehicle
```

So SQL Server told us:

```
Session 73
    |
    | SELECT
    ▼
```

```
⏳ SUSPENDED
    |
    | waiting for lock
    ▼
Session 71
    |
    └── BLOCKING
```

And the request was waiting for almost **59 seconds**.

This is exactly the kind of event our AI monitoring system needs to detect.

Next: `blocking.py`

Now we're going to separate **blocking detection** from general request monitoring.

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\blocking.py
```

Put this in it:

Python Run

```python
from db_connection import get_connection


def get_blocking_requests():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            r.session_id AS blocked_session_id,
            r.blocking_session_id,
            r.status,
            r.wait_type,
            r.wait_time,
            r.cpu_time,
            r.total_elapsed_time,
            r.reads,
            r.writes,
            r.logical_reads,
            DB_NAME(r.database_id) AS database_name,
            s.login_name,
            s.host_name,
```

```
            s.program_name,
            t.text AS sql_text
        FROM sys.dm_exec_requests r
        INNER JOIN sys.dm_exec_sessions s
            ON r.session_id = s.session_id
        CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
        WHERE r.blocking_session_id <> 0
          AND s.is_user_process = 1
        ORDER BY r.wait_time DESC;
        """

        cursor.execute(query)

        rows = cursor.fetchall()

        blocking_requests = []

        for row in rows:
            blocking_requests.append({
                "blocked_session_id": row.blocked_session_id,
                "blocking_session_id": row.blocking_session_id,
                "status": row.status,
                "wait_type": row.wait_type,
                "wait_time_ms": row.wait_time,
                "cpu_time_ms": row.cpu_time,
                "elapsed_time_ms": row.total_elapsed_time,
                "reads": row.reads,
                "writes": row.writes,
                "logical_reads": row.logical_reads,
                "database_name": row.database_name,
                "login_name": row.login_name,
                "host_name": row.host_name,
                "program_name": row.program_name,
                "sql_text": row.sql_text
            })

        return {
            "metric": "blocking",
            "count": len(blocking_requests),
            "blocking_requests": blocking_requests,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":
    result = get_blocking_requests()

    print(f"Blocking request count: {result['count']}")
```

```
    for blocking in result["blocking_requests"]:
        print(blocking)
```

Important difference from `requests.py`

Our `requests.py` asks:

> **What requests are currently executing?**

`blocking.py` asks:

> **Are any requests currently blocked by another session?**

The key filter is:

```
    SQL

WHERE r.blocking_session_id <> 0
```

That's the first primitive of our future monitoring Rule Engine.

Test it

You already rolled back the transaction, so right now we expect:

```
    PowerShell

python blocking.py
```

to produce:

```
Blocking request count: 0
```

That's correct.

Then, if we reproduce the blocking scenario and run:

```
    PowerShell

python blocking.py
```

while the victim is waiting, we should see something like:

```
Blocking request count: 1

{
    'blocked_session_id': 73,
    'blocking_session_id': 71,
    'status': 'suspended',
    'wait_type': 'LCK_M_IS',
    'wait_time_ms': 12345,
    'database_name': 'Vehicle',
    ...
}
```

But there's a limitation

Notice something important.

We know:

```
blocked_session_id = 73
blocking_session_id = 71
```

But we currently only have the SQL text of **session 73**, the blocked request.

We don't yet know:

> **What exactly is session 71 doing?**

That's critical.

Our eventual AI diagnosis needs both sides:

```
BLOCKING CHAIN

Session 71
   │
   ├── Login
   ├── Host
   ├── Program
   ├── Transaction
   └── SQL
        │
        ▼
     BLOCKING
        │
        ▼
Session 73
```

```
    │
    ├── SQL
    ├── Wait type
    ├── Wait duration
    └── Database
```

So after we confirm `blocking.py` works, we'll improve it to retrieve **the blocker information as well**.

That's when the output starts looking like a real DBA monitoring event:

```
🚨 BLOCKING DETECTED

Database: Vehicle

BLOCKER
  Session: 71
  Login: Moosa
  SQL: ...

VICTIM
  Session: 73
  SQL: SELECT TOP 1 ...
  Wait: LCK_M_IS
  Duration: 59 seconds
```

And **after that**, we'll start thinking about the Rule Engine.

For now, create `blocking.py` and run it once with the transaction rolled back.

Send me the output.

Actually i have created 2 blocks adn see is it work. I got the both the info. Here ithe output

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe blocking.py
Blocking request count: 2
{'blocked_session_id': 73, 'blocking_session_id': 71, 'status': 'suspended', 'wait_type': 'LCK_M_IS', 'wait_time_ms': 11808, 'cpu_time_ms': 0, 'elapsed_time_ms': 11808, 'reads': 0, 'writes': 0, 'logical_reads': 14, 'database_name': 'Vehicle',

Show more

Exactly! 🔥 This is **much better** because you created two blocked requests, and our collector detected both.

Your output proves:

```
Blocking request count: 2
```

And both are being blocked by the same session:

```
Session 71
    |
    |──────────────► Session 73
    |                  LCK_M_IS
    |
    └──────────────► Session 52
                       LCK_M_IS
```

So we have a **blocking chain / blocking tree** rather than just a single blocked request.

What your monitor currently knows

Item	Session 73	Session 52
Status	suspended	suspended
Blocker	**71**	**71**
Wait	LCK_M_IS	LCK_M_IS
Wait time	11.8 sec	5.6 sec
Database	Vehicle	Vehicle
CPU	0 ms	0 ms

This is exactly the kind of raw evidence our AI layer will eventually consume.

But I want to improve `blocking.py` before moving on

Currently we know:

```
blocked_session_id → blocking_session_id
```

but we don't yet collect information **about session 71 itself**.

For a DBA, that's the critical question:

> **"Okay, session 71 is blocking them. What is session 71 doing?"**

So our next version should return both:

```
BLOCKER
   ↓
VICTIMS
```

For example:

```
{
    "blocking_session_id": 71,

    "blocker": {
        "login_name": "...",
        "host_name": "...",
        "program_name": "...",
        "status": "...",
        "sql_text": "..."
    },

    "victims": [
        {
            "session_id": 73,
            "wait_type": "LCK_M_IS",
            "wait_time_ms": 11808,
            "sql_text": "SELECT TOP 1 ..."
        },
        {
            "session_id": 52,
            "wait_type": "LCK_M_IS",
            "wait_time_ms": 5629,
            "sql_text": "SELECT TOP 1 ..."
        }
    ]
}
```

That gives us a much more useful **blocking event**.

One more DBA-level concept we'll capture

There's a subtle difference between:

```
blocking_session_id
```

and the **root blocker**.

Imagine:

```
71
 ↓
73
 ↓
85
 ↓
91
```

Session 71 may be the **root blocker**, while 73 and 85 are themselves blocking downstream sessions.

Our eventual monitor should be able to build this tree.

That's going to be very useful for the AI diagnosis component.

So our progression is now

```
✅ db_connection.py
        ↓
✅ server_health.py
        ↓
✅ cpu.py
        ↓
✅ memory.py
        ↓
✅ sessions.py
        ↓
✅ requests.py
        ↓
✅ blocking.py
        ↓
➡️ Enhanced blocking / blocker details
        ↓
➡️ Wait statistics
        ↓
➡️ I/O
        ↓
➡️ Query performance
        ↓
➡️ Rule engine
        ↓
```

```
➡️ Alerting
        ↓
➡️ AI investigation
```

We're actually building this in a sensible DBA-first order.

For now

Keep your blocking test running if you still have the two sessions open. If you have already rolled them back, that's fine too.

The next step is to upgrade `blocking.py` so it identifies the **blocker itself**, not just the victims.

I'll give you that version next.

please go ahead

Absolutely. Let's upgrade `blocking.py` so it reports **who is blocking whom and what the blocker is doing**.

1. Replace `blocking.py`

Use this version:

Python Run

```python
from db_connection import get_connection


def get_blocking_requests():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            r.session_id AS blocked_session_id,
            r.blocking_session_id,

            r.status,
            r.wait_type,
            r.wait_time,
            r.cpu_time,
            r.total_elapsed_time,
```

```
        r.reads,
        r.writes,
        r.logical_reads,

        DB_NAME(r.database_id) AS database_name,

        blocked.login_name AS blocked_login,
        blocked.host_name AS blocked_host,
        blocked.program_name AS blocked_program,

        blocked_sql.text AS blocked_sql_text,

        blocker.status AS blocker_status,
        blocker.login_name AS blocker_login,
        blocker.host_name AS blocker_host,
        blocker.program_name AS blocker_program,

        blocker_sql.text AS blocker_sql_text

    FROM sys.dm_exec_requests r

    INNER JOIN sys.dm_exec_sessions blocked
        ON r.session_id = blocked.session_id

    LEFT JOIN sys.dm_exec_sessions blocker
        ON r.blocking_session_id = blocker.session_id

    OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) blocked_sql

    OUTER APPLY sys.dm_exec_sql_text(
        blocker.most_recent_sql_handle
    ) blocker_sql

    WHERE r.blocking_session_id <> 0
      AND blocked.is_user_process = 1

    ORDER BY r.wait_time DESC;
    """

    cursor.execute(query)

    rows = cursor.fetchall()

    blocking_requests = []

    for row in rows:

        blocking_requests.append({
            "blocked_session": {
                "session_id": row.blocked_session_id,
                "login_name": row.blocked_login,
                "host_name": row.blocked_host,
```

```
                    "program_name": row.blocked_program,
                    "status": row.status,
                    "wait_type": row.wait_type,
                    "wait_time_ms": row.wait_time,
                    "cpu_time_ms": row.cpu_time,
                    "elapsed_time_ms": row.total_elapsed_time,
                    "reads": row.reads,
                    "writes": row.writes,
                    "logical_reads": row.logical_reads,
                    "sql_text": row.blocked_sql_text
                },

                "blocking_session": {
                    "session_id": row.blocking_session_id,
                    "login_name": row.blocker_login,
                    "host_name": row.blocker_host,
                    "program_name": row.blocker_program,
                    "status": row.blocker_status,
                    "sql_text": row.blocker_sql_text
                },

                "database_name": row.database_name
            })

        return {
            "metric": "blocking",
            "count": len(blocking_requests),
            "blocking_requests": blocking_requests,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":

    result = get_blocking_requests()

    print(f"Blocking request count: {result['count']}")

    for blocking in result["blocking_requests"]:

        print("\n--- BLOCKING EVENT ---")

        print("Database:", blocking["database_name"])

        print(
            "Blocked session:",
            blocking["blocked_session"]["session_id"]
        )
```

```
        print(
            "Blocking session:",
            blocking["blocking_session"]["session_id"]
        )

        print(
            "Wait type:",
            blocking["blocked_session"]["wait_type"]
        )

        print(
            "Wait time:",
            blocking["blocked_session"]["wait_time_ms"],
            "ms"
        )

        print(
            "Blocked SQL:",
            blocking["blocked_session"]["sql_text"]
        )

        print(
            "Blocker login:",
            blocking["blocking_session"]["login_name"]
        )

        print(
            "Blocker program:",
            blocking["blocking_session"]["program_name"]
        )

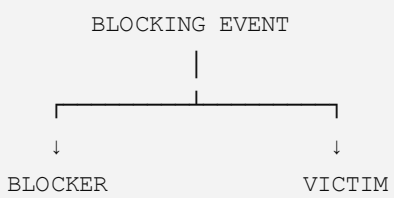
        print(
            "Blocker SQL:",
            blocking["blocking_session"]["sql_text"]
```

2. What changed?

Previously we had:

```
Blocked session
      ↓
Blocking session ID
```

Now we have:



```
          |                  |
      Session 71         Session 73
          |                  |
      Login               Login
      Program             Program
      SQL                 SQL
                            |
                        Wait type
                        Wait time
```

This is much closer to the data structure our future AI engine needs.

One particularly important field

Python Run

```
blocker_sql
```

We are attempting to retrieve the blocker's most recent SQL handle.

This is useful because a blocker can be **sleeping** while still holding locks because it has an open transaction.

That's a very common real-world SQL Server blocking scenario.

---

3. Test it

If you've already rolled back the blocking transaction, first run:

PowerShell

```
python blocking.py
```

Expected:

```
Blocking request count: 0
```

Then you can reproduce your blocking test again and run:

PowerShell

```
python blocking.py
```

You should get something along the lines of:

```
Blocking request count: 2

--- BLOCKING EVENT ---

Database: Vehicle
Blocked session: 73
Blocking session: 71
Wait type: LCK_M_IS
Wait time: 12000 ms

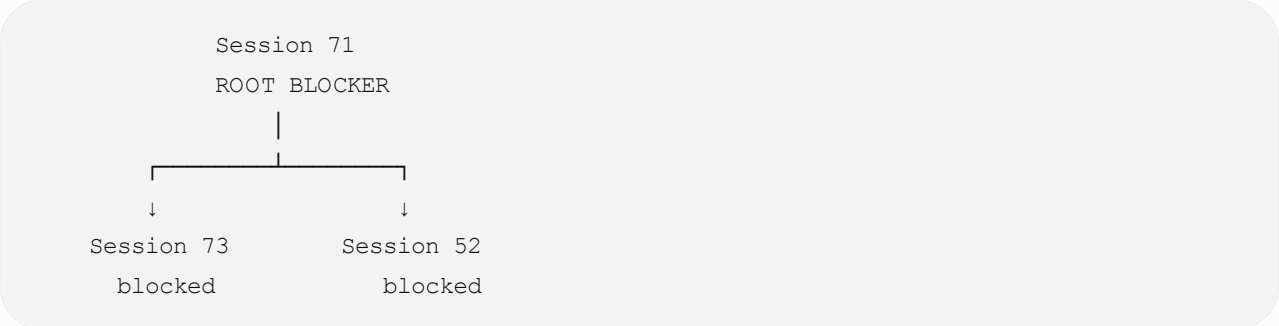
Blocked SQL: SELECT TOP 1 ...

Blocker login: MOOSA-DELLLAPTO\Moosa
Blocker program: Microsoft SQL Server Management Studio
Blocker SQL: ...
```

The exact blocker SQL text may vary depending on what SQL Server has retained for that session.

One thing I want you to watch carefully

You created **two victims with one blocker**:



Our current output will give us **two blocking events**.

Later, rather than treating those as two unrelated incidents, we'll consolidate them into:

```
Blocking Incident #1

Root blocker: 71

Victims:
    73
    52

Total victims: 2
```

That's an important step toward intelligent monitoring.

Run it now

PowerShell

```
python blocking.py
```

If you still have the blocking scenario active, send me the output.

If you've already rolled it back, just run it anyway and show me the `Blocking request count`. Either result is fine.

i am getting error on the query. one column is not found. I am using sql server 2022 develeopededtion

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe blocking.py
Traceback (most recent call last):
  File "D:\Python\AI_SQL_Monitor\monitoring\blocking.py", line 109, in <module>
    result = get_blocking_requests()
             ^^^^^^^^^^^^^^^^^^^^^^
  File "D:\Python\AI_SQL_Monitor\monitoring\blocking.py",

Show more

Yes — that's my mistake. The problem is **not SQL Server 2022** or ODBC Driver 17.

I used:

SQL

```
blocker.most_recent_sql_handle
```

as if `sys.dm_exec_sessions` exposed that column. It **doesn't**. `most_recent_sql_handle` isn't a column of `sys.dm_exec_sessions`.

Let's correct the query rather than work around the error.

What we need to do

For the **blocked request**, we have:

SQL

```
r.sql_handle
```

so getting its SQL text is straightforward.

For the **blocker**, however, we need a different approach. A blocking session may have **no active request at all**—especially when it is a sleeping session holding an open transaction. Therefore, we shouldn't assume we can always retrieve its SQL text from `sys.dm_exec_requests`.

For this stage, let's first reliably collect the **blocker's session information** and the victim's SQL.

Replace your `blocking.py` with this corrected version:

Python Run

```
from db_connection import get_connection


def get_blocking_requests():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            r.session_id AS blocked_session_id,
            r.blocking_session_id,

            r.status,
            r.wait_type,
            r.wait_time,
            r.cpu_time,
            r.total_elapsed_time,
            r.reads,
            r.writes,
            r.logical_reads,

            DB_NAME(r.database_id) AS database_name,

            blocked.login_name AS blocked_login,
            blocked.host_name AS blocked_host,
            blocked.program_name AS blocked_program,

            blocked_sql.text AS blocked_sql_text,
```

```
            blocker.status AS blocker_status,
            blocker.login_name AS blocker_login,
            blocker.host_name AS blocker_host,
            blocker.program_name AS blocker_program

        FROM sys.dm_exec_requests r

        INNER JOIN sys.dm_exec_sessions blocked
            ON r.session_id = blocked.session_id

        LEFT JOIN sys.dm_exec_sessions blocker
            ON r.blocking_session_id = blocker.session_id

        OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) blocked_sql

        WHERE r.blocking_session_id <> 0
          AND blocked.is_user_process = 1

        ORDER BY r.wait_time DESC;
        """

        cursor.execute(query)

        rows = cursor.fetchall()

        blocking_requests = []

        for row in rows:

            blocking_requests.append({
                "blocked_session": {
                    "session_id": row.blocked_session_id,
                    "login_name": row.blocked_login,
                    "host_name": row.blocked_host,
                    "program_name": row.blocked_program,
                    "status": row.status,
                    "wait_type": row.wait_type,
                    "wait_time_ms": row.wait_time,
                    "cpu_time_ms": row.cpu_time,
                    "elapsed_time_ms": row.total_elapsed_time,
                    "reads": row.reads,
                    "writes": row.writes,
                    "logical_reads": row.logical_reads,
                    "sql_text": row.blocked_sql_text
                },

                "blocking_session": {
                    "session_id": row.blocking_session_id,
                    "login_name": row.blocker_login,
                    "host_name": row.blocker_host,
                    "program_name": row.blocker_program,
                    "status": row.blocker_status
```

```
                },

                "database_name": row.database_name
            })

        return {
            "metric": "blocking",
            "count": len(blocking_requests),
            "blocking_requests": blocking_requests,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":

    result = get_blocking_requests()

    print(f"Blocking request count: {result['count']}")

    for blocking in result["blocking_requests"]:

        print("\n--- BLOCKING EVENT ---")

        print("Database:", blocking["database_name"])

        print(
            "Blocked session:",
            blocking["blocked_session"]["session_id"]
        )

        print(
            "Blocking session:",
            blocking["blocking_session"]["session_id"]
        )

        print(
            "Wait type:",
            blocking["blocked_session"]["wait_type"]
        )

        print(
            "Wait time:",
            blocking["blocked_session"]["wait_time_ms"],
            "ms"
        )

        print(
            "Blocked SQL:",
            blocking["blocked_session"]["sql_text"]
```

```
        )

        print(
            "Blocker login:",
            blocking["blocking_session"]["login_name"]
        )

        print(
            "Blocker program:",
            blocking["blocking_session"]["program_name"]
        )

        print(
            "Blocker status:",
            blocking["blocking_session"]["status"]
```

Now test it

If you've already rolled back your blocking transaction:

PowerShell

```
python blocking.py
```

You should get:

```
Blocking request count: 0
```

Then recreate your blocking test and run it while the victim is waiting:

PowerShell

```
python blocking.py
```

You should get something like:

```
--- BLOCKING EVENT ---

Database: Vehicle
Blocked session: 73
Blocking session: 71
Wait type: LCK_M_IS
Wait time: 12000 ms
Blocked SQL: SELECT TOP 1 ...

Blocker login: MOOSA-DELLLAPTO\Moosa
Blocker program: Microsoft SQL Server Management Studio - Query
Blocker status: sleeping
```

And **this is actually a valuable result**: if the blocker says `sleeping`, that demonstrates the classic DBA situation where a session is not currently executing a request but can still be holding locks because of an open transaction.

One correction to our architecture

I don't want to rush into trying to get "blocker SQL text" with another shortcut. We should next investigate **transactions and locks** properly.

Our next monitoring component should therefore answer:

> **Why is Session 71 able to block Session 73? What lock does it hold, on which object, and is there an open transaction?**

That means we'll look at:

```
sys.dm_tran_locks
        +
sys.dm_tran_session_transactions
        +
sys.dm_exec_requests
        +
sys.dm_exec_sessions
```

That will give us a much stronger blocking detector than simply looking at `blocking_session_id`.

Run the corrected `blocking.py` first.

Now it worked. Here ithe ouptut

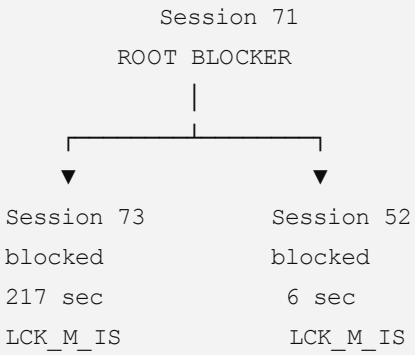
(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe blocking.py
Blocking request count: 2

--- BLOCKING EVENT ---
Database: Vehicle
Blocked session: 73
Blocking session: 71
Wait type: LCK_M_IS

Show more

Excellent. 🔥 This is exactly the behavior we wanted to capture.

Your monitor has now detected **two victims with the same blocker**:



The especially interesting part is:

```
Blocker status: sleeping
```

That is a very realistic SQL Server scenario: **the blocker doesn't have to be actively running a query to continue holding locks**. An open transaction can keep the locks while the session is sleeping.

Before we go further: clean up the test

Make sure the blocking transaction in Session 71 is rolled back:

SQL

```
ROLLBACK TRANSACTION;
```

Then verify:

PowerShell

```
python blocking.py
```

You should get:

```
Blocking request count: 0
```

Next step: Understand the blocker

Now I want to take one step deeper rather than immediately adding another Python file.

We know:

```
blocking_session_id = 71
```

But we want to answer:

> **What transaction does session 71 have open, and what locks does it hold?**

This is much more valuable for our AI monitor.

We'll investigate two DMVs:

1. `sys.dm_tran_session_transactions`

This tells us whether the session has an active transaction.

2. `sys.dm_tran_locks`

This tells us about locks currently held/requested.

The relationship becomes:

```
Session 71
    |
    ├── Active transaction?
    |
    └── Locks
          |
          └── Electric_Vehicle_Population_Data
```

Then the victim:

```
Session 73
    |
    └── Request
          |
          └── LCK_M_IS
```

Now our eventual diagnosis can become much more intelligent:

> **Session 71 has an open transaction holding an incompatible lock on the Vehicle table. Sessions 73 and 52 are waiting for compatible access.**

That's far better than simply:

> "Blocking detected."

Run this in SSMS

First recreate your blocking scenario exactly as before.

Then, **while the blocking is active**, run:

```sql
SELECT
    st.session_id,
    st.transaction_id,
    st.is_local,
    st.is_enlisted,
    st.open_transaction_count
FROM sys.dm_tran_session_transactions st
WHERE st.session_id = 71;
```

Then run:

```sql
SELECT
    tl.request_session_id,
    tl.resource_type,
    tl.resource_database_id,
    DB_NAME(tl.resource_database_id) AS database_name,
    tl.resource_description,
    tl.request_mode,
    tl.request_status
FROM sys.dm_tran_locks tl
WHERE tl.request_session_id IN (71, 73, 52)
ORDER BY tl.request_session_id;
```

Why I'm asking for the actual output

I don't want to guess what SQL Server is returning on your Developer Edition instance.

We'll use **your actual lock information** to design the next collector correctly.

After you've captured the results, you can roll back the transaction again.

Then send me the two result sets.

From there we'll build the next component—**transaction/lock monitoring**—and eventually combine it with `blocking.py` rather than creating isolated collectors forever.

We're now moving from:

```
CPU
Memory
```

```
Sessions
```

into:

```
Locks
   ↓
Transactions
   ↓
Blocking chains
   ↓
Root-cause evidence
```

That's where this project starts becoming a real **SQL Server monitoring system**, rather than just a collection of DMV scripts.

71 183185 1 0 1

52 DATABASE 5 Vehicle
S GRANT
52 OBJECT 5 Vehicle
IS WAIT
71 OBJECT 5 Vehicle
X GRANT
71 OBJECT 5 Vehicle
X GRANT
73 OBJECT 5 Vehicle

Show more

Yes — and **no more output requests from you for this step**. You have given us enough. 👍

Your results are actually very useful. Let's interpret them and move forward.

What SQL Server is telling us

Session **71**:

```
transaction_id       = 183185
open_transaction_count = 1
```

So we have confirmed:

```
Session 71
    |
```

```
    └── Open transaction
          │
          └── Transaction 183185
```

And the locks show:

```
Session 71 → OBJECT → X → GRANT
```

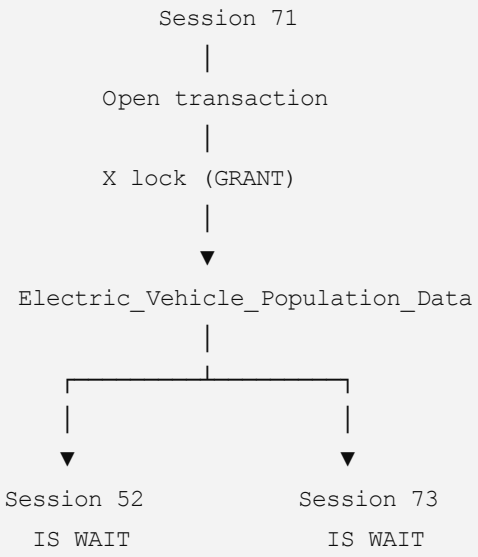
That means session 71 has an **exclusive (X) lock** on the table.

Meanwhile:

```
Session 52 → OBJECT → IS → WAIT
Session 73 → OBJECT → IS → WAIT
```

Both sessions are waiting for an **Intent Shared (IS)** lock.

So our blocking picture is now:



That is excellent evidence for our monitoring engine.

Now let's create `locks.py`

We're ready to turn this into another reusable collector.

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\locks.py
```

Use:

Python Run

```python
from db_connection import get_connection


def get_locks():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        query = """
        SELECT
            tl.request_session_id AS session_id,
            tl.resource_type,
            tl.resource_database_id,
            DB_NAME(tl.resource_database_id) AS database_name,
            tl.request_mode,
            tl.request_status,
            st.transaction_id,
            st.open_transaction_count
        FROM sys.dm_tran_locks tl

        LEFT JOIN sys.dm_tran_session_transactions st
            ON tl.request_session_id = st.session_id

        WHERE tl.request_session_id <> @@SPID

        ORDER BY
            tl.request_session_id,
            tl.request_status DESC;
        """

        cursor.execute(query)

        rows = cursor.fetchall()

        locks = []

        for row in rows:
            locks.append({
                "session_id": row.session_id,
                "resource_type": row.resource_type,
                "database_name": row.database_name,
                "request_mode": row.request_mode,
                "request_status": row.request_status,
                "transaction_id": row.transaction_id,
                "open_transaction_count": row.open_transaction_count
            })

        return {
            "metric": "locks",
```

```
            "count": len(locks),
            "locks": locks,
            "status": "ok"
        }

    finally:
        connection.close()


if __name__ == "__main__":

    result = get_locks()

    print(f"Lock count: {result['count']}")

    for lock in result["locks"]:
        print(lock)
```

Why `locks.py` is separate

We now have different monitoring responsibilities:

```
cpu.py
    → CPU

memory.py
    → Memory

sessions.py
    → Connections

requests.py
    → Currently executing work

blocking.py
    → Blocked requests

locks.py
    → Locks and transactions
```

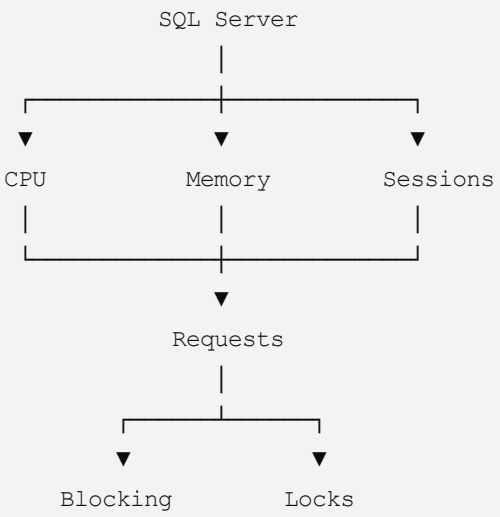
That's a clean separation.

But here's where the project changes direction

We have now collected enough **raw SQL Server telemetry** to start thinking about our architecture.

We shouldn't keep creating DMV files indefinitely.

Our monitoring layer is becoming:



The next major stage should be **not another random DMV**.

It should be:

Monitoring Collector

We'll create something like:

```
monitoring/
│
├── db_connection.py
├── server_health.py
│
├── cpu.py
├── memory.py
├── sessions.py
├── requests.py
├── blocking.py
├── locks.py
│
└── collector.py       ← next major component
```

`collector.py` will call the individual collectors and produce **one monitoring snapshot**.

For example:

JSON

```
{
    "server": "Moosa-DELLLaptop",
    "database": "Vehicle",
    "cpu": {...},
    "memory": {...},
```

```
    "sessions": {...},
    "requests": {...},
    "blocking": {...},
    "locks": {...}
}
```

That is a major milestone because **this is the data that our future Rule Engine and AI layer will consume**.

So I recommend we stop adding individual metrics for the moment and build `collector.py` next.

And don't worry about sending me more SSMS output. **From this point, I'll give you the code and explain what we're building; you can simply run it and tell me whether it works or paste an error if one occurs.**

want to add more info to my project scope. Please update it

Current Development Environment

My laptop has:

- Dell laptop
- Intel i9 processor
- 32 GB RAM
- No dedicated GPU

Show more

Memory updated

Absolutely. I've incorporated these decisions into the project scope, including the **zero-cost local-first approach**, **PostgreSQL + pgvector**, and **provider-independent LLM architecture**.

And yes — let's proceed with `collector.py`.

Step 5 — Build `collector.py`

We've now built these individual collectors:

```
monitoring/
|
```

```
├── db_connection.py
├── server_health.py
├── cpu.py
├── memory.py
├── sessions.py
├── requests.py
├── blocking.py
├── locks.py
└── collector.py       ← NEW
```

The purpose of `collector.py` is simple:

> **Call all our monitoring components and produce one complete snapshot.**

Instead of doing this:

```
main program
   ├── call cpu.py
   ├── call memory.py
   ├── call sessions.py
   ├── call requests.py
   ├── call blocking.py
   └── call locks.py
```

we'll eventually do:

```
main program
      |
      ▼
 collector.py
      |
      ├── CPU
      ├── Memory
      ├── Sessions
      ├── Requests
      ├── Blocking
      └── Locks
      |
      ▼
Monitoring Snapshot
```

Create `collector.py`

Put it here:

```
D:\Python\AI_SQL_Monitor\monitoring\collector.py
```

Use:

Python

Run

```python
from server_health import get_server_health
from cpu import get_cpu_usage
from memory import get_memory_status
from sessions import get_user_sessions
from requests import get_active_requests
from blocking import get_blocking_requests
from locks import get_locks


def collect_snapshot():
    snapshot = {
        "server_health": get_server_health(),
        "cpu": get_cpu_usage(),
        "memory": get_memory_status(),
        "sessions": get_user_sessions(),
        "requests": get_active_requests(),
        "blocking": get_blocking_requests(),
        "locks": get_locks()
    }

    return snapshot


if __name__ == "__main__":
    snapshot = collect_snapshot()

    print("\n========== SQL SERVER MONITORING SNAPSHOT ==========\n")

    print("SERVER HEALTH")
    print(snapshot["server_health"])

    print("\nCPU")
    print(snapshot["cpu"])

    print("\nMEMORY")
    print(snapshot["memory"])

    print("\nSESSIONS")
    print(snapshot["sessions"])

    print("\nREQUESTS")
    print(snapshot["requests"])

    print("\nBLOCKING")
    print(snapshot["blocking"])

    print("\nLOCKS")
    print(snapshot["locks"])
```

One thing to check

Your existing `server_health.py` function needs to be named:

```python
get_server_health()
```

If you currently have a different function name, **don't change it yet**. Just tell me the error when you run `collector.py`, and we'll align it.

Run it

From:

```
(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
```

run:

```powershell
python collector.py
```

Because you rolled back the blocking transaction, ideally you'll see:

```
BLOCKING
{'metric': 'blocking', 'count': 0, ...}
```

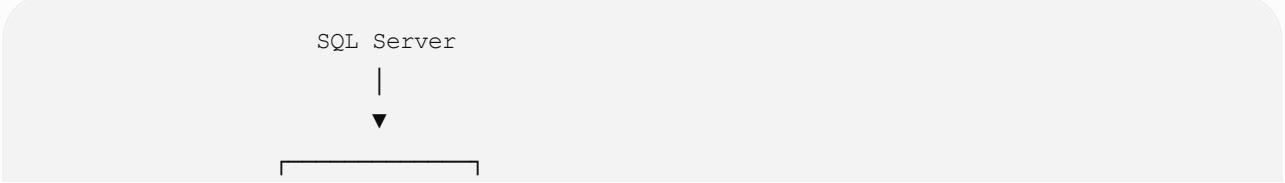
and probably some active sessions/requests caused by the collector itself.

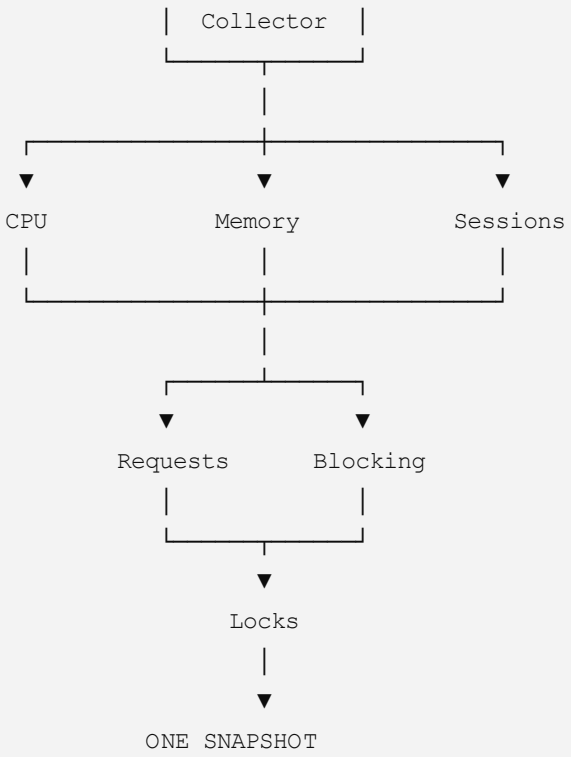
Why this is an important milestone

Before `collector.py`, our architecture was basically:

```
Several independent Python scripts
```

Now we're creating:





That **one snapshot** is what we'll eventually send to the next layer.

And this is where your newly added architecture decisions become important.

Eventually:

```
SQL Server
    |
    ▼
Collector
    |
    ▼
Monitoring Snapshot
    |
    ▼
Rule Engine
    |
    ├── Normal
    ├── Warning
    └── Critical
            |
            ▼
       Investigation
            |
            ▼
        RAG Knowledge
            |
       PostgreSQL +
          pgvector
            |
            ▼
```

```
            LLM
             |
       OpenRouter initially
             |
             ▼
       AI Diagnosis
```

Notice that **the LLM isn't involved yet**.

That's intentional.

We first build reliable telemetry and deterministic monitoring. Then we'll introduce the AI layer.

Next action

Run:

```
PowerShell

python collector.py
```

You don't need to paste the entire output unless there's an error. **Just tell me whether it completed successfully or give me the error**, and we'll move to the next architectural step.

all went good. I am able to see all step output from teh collect.py execution. WE can proceed further

Excellent. ✅ We now have the **first complete monitoring pipeline** working.

I suggest we move to the next major stage: **persisting monitoring snapshots**.

Right now:

```
SQL Server
    ↓
individual collectors
    ↓
collector.py
    ↓
snapshot
    ↓
printed on screen
```

But our application eventually needs historical data:

```
10:00 → CPU 5%
10:01 → CPU 8%
10:02 → CPU 73%
10:03 → blocking detected
10:04 → blocking cleared
```

Without persistence, the AI cannot later answer things like:

> "What happened to this SQL Server during the last 30 minutes?"

Step 6 — Add a Monitoring Repository

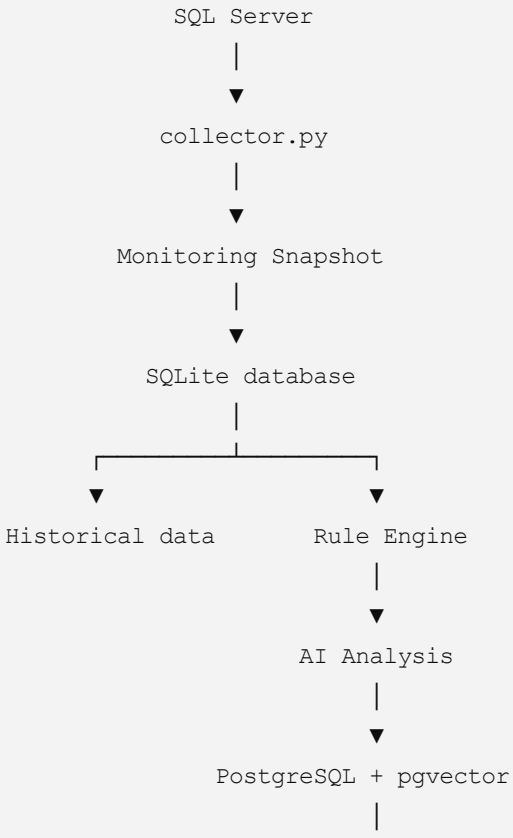
Because your project is intentionally **local and zero-cost**, I recommend we **do not put monitoring telemetry into PostgreSQL yet**.

Keep the architecture clean:

- **SQL Server** → monitored production/operational database
- **PostgreSQL + pgvector** → later RAG/knowledge store
- **Monitoring telemetry** → initially local storage

For the first implementation, we'll use **SQLite** for telemetry.

So the architecture becomes:



▼

This also keeps PostgreSQL available for its intended purpose instead of mixing two unrelated data responsibilities.

Create `monitoring_repository.py`

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\monitoring_repository.py
```

Start with this:

Python Run

```python
import sqlite3
import json
from datetime import datetime


DATABASE = "monitoring.db"


def get_connection():
    return sqlite3.connect(DATABASE)


def initialize_database():
    connection = get_connection()

    try:
        cursor = connection.cursor()

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS monitoring_snapshots (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                collected_at TEXT NOT NULL,
                snapshot_json TEXT NOT NULL
            )
        """)

        connection.commit()

    finally:
        connection.close()


def save_snapshot(snapshot):
```

```
    connection = get_connection()

    try:
        cursor = connection.cursor()

        collected_at = datetime.now().isoformat()

        cursor.execute(
            """
            INSERT INTO monitoring_snapshots
            (
                collected_at,
                snapshot_json
            )
            VALUES (?, ?)
            """,
            (
                collected_at,
                json.dumps(snapshot, default=str)
            )
        )

        connection.commit()

    finally:
        connection.close()


def get_recent_snapshots(limit=10):
    connection = get_connection()

    try:
        cursor = connection.cursor()

        cursor.execute(
            """
            SELECT
                id,
                collected_at,
                snapshot_json
            FROM monitoring_snapshots
            ORDER BY id DESC
            LIMIT ?
            """,
            (limit,)
        )

        rows = cursor.fetchall()

        snapshots = []

        for row in rows:
```

```
            snapshots.append({
                "id": row[0],
                "collected_at": row[1],
                "snapshot": json.loads(row[2])
            })

        return snapshots

    finally:
        connection.close()


if __name__ == "__main__":

    initialize_database()
```

Test it

Run:

PowerShell

```
python monitoring_repository.py
```

You should get:

```
Monitoring repository initialized successfully.
```

And a new file should appear:

```
monitoring/
│
├── monitoring.db
├── db_connection.py
├── server_health.py
├── cpu.py
├── memory.py
├── sessions.py
├── requests.py
├── blocking.py
├── locks.py
├── collector.py
└── monitoring_repository.py
```

Don't worry about the database being SQLite

This is **not replacing PostgreSQL**.

We're deliberately separating the two purposes:

```
SQLite
└── Monitoring telemetry/history

PostgreSQL + pgvector
└── AI/RAG knowledge
    ├── documents
    ├── chunks
    ├── embeddings
    └── metadata
```

Later, if we discover that PostgreSQL is better for the telemetry as well, we can change the repository layer without changing the collectors.

That's why we created a **repository abstraction** rather than putting SQLite code directly inside `collector.py`.

Then we connect the collector

Once `monitoring_repository.py` works, our next change will be small:

```
collector.py
     │
     ├── collect snapshot
     │
     └── save snapshot
              │
              ▼
        monitoring.db
```

Then we'll add a simple scheduler so the monitor can continuously collect:

```
Every 30 seconds
       ↓
collector
       ↓
save snapshot
       ↓
wait
       ↓
repeat
```

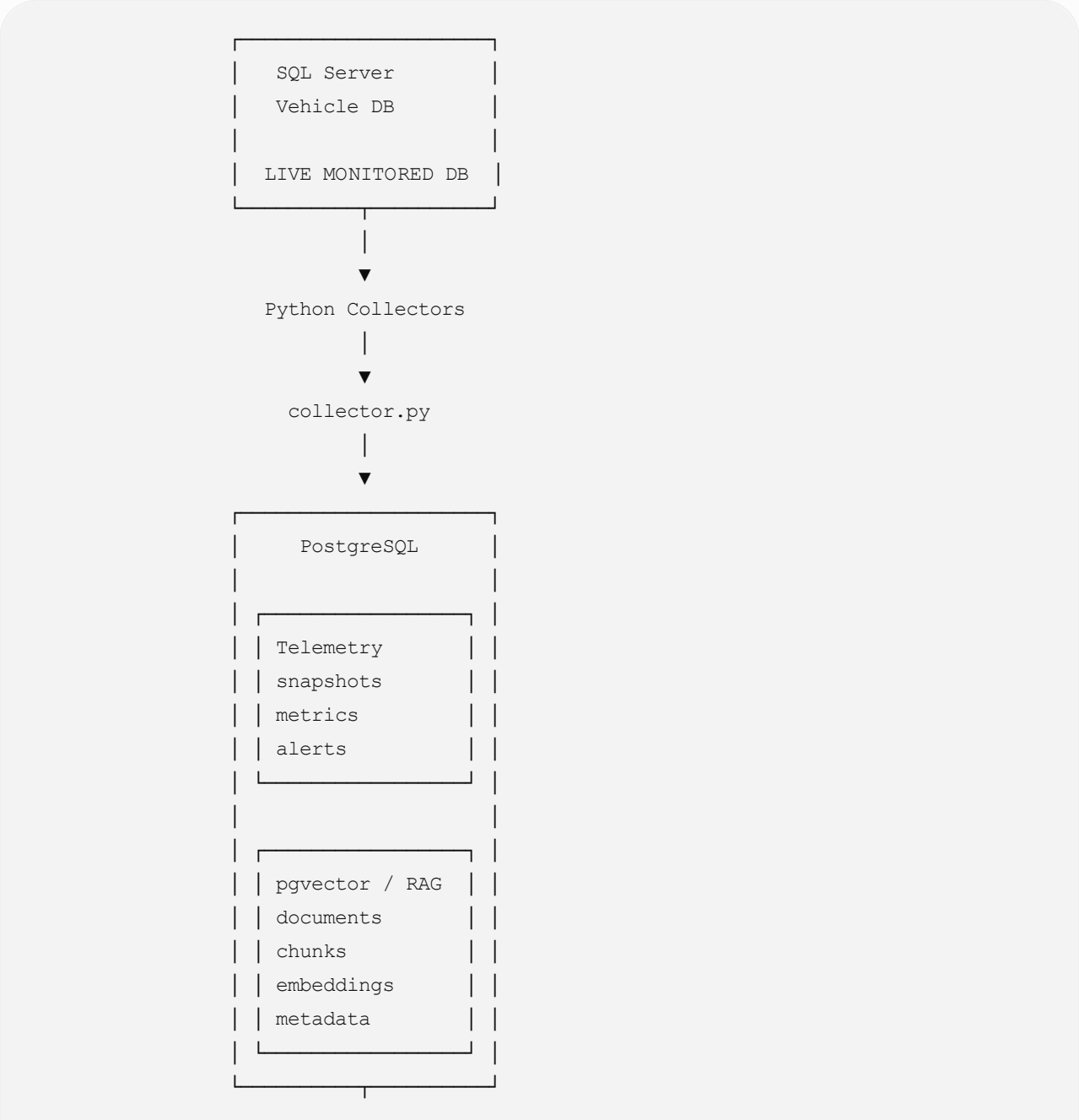
Run `python monitoring_repository.py` **first.** If it completes successfully, just tell me `done` and we'll wire it into `collector.py.

NO i dont' want sqllite. it will be slow and hard to maintain. will either choose post gres or sql server for the telemetry

Agreed. **Let's not introduce SQLite.** For this project, PostgreSQL is the better choice.

In fact, given your scope, I would choose **PostgreSQL for telemetry as well as RAG**, while keeping the monitored SQL Server completely separate.

Revised architecture



```
                    |
                    ▼
               Rule Engine
                    |
                    ▼
             AI Investigation
                    |
                    ▼
              OpenRouter LLM
```

This is cleaner than introducing a third database.

One important distinction

We'll have **two completely different databases**:

SQL Server — system under monitoring

```
localhost
└── Vehicle
    └── Electric_Vehicle_Population_Data
```

We don't modify this for our monitoring application.

PostgreSQL — monitoring platform database

We'll eventually have something like:

```
AI_SQL_Monitor
│
├── telemetry
│
├── alerts
│
├── incidents
│
├── monitoring_rules
│
├── documents
│
├── document_chunks
│
└── embeddings
```

So PostgreSQL becomes the **backend database of our monitoring application**.

And `pgvector` is used specifically where we need vector search.

One correction to our previous step

Don't create `monitoring_repository.py` with SQLite.

We're abandoning that implementation entirely.

We haven't lost anything—we're simply making the architecture better before we start persisting data.

What I recommend next

Before writing the PostgreSQL repository code, let's establish the PostgreSQL connection properly.

We'll create:

```
monitoring/
│
├── db_connection.py       ← SQL Server
│
├── pg_connection.py       ← PostgreSQL
│
├── server_health.py
├── cpu.py
├── memory.py
├── sessions.py
├── requests.py
├── blocking.py
├── locks.py
│
└── collector.py
```

This makes the distinction extremely clear:

```
db_connection.py
       ↓
SQL Server
       ↓
collect information


pg_connection.py
       ↓
PostgreSQL
       ↓
store information
```

Don't mix these connections.

That's especially important later when we add the AI/RAG layer.

And PostgreSQL won't just be a "vector database"

Your original scope says PostgreSQL + pgvector is the initial vector database.

I'd slightly refine that architecture:

> **PostgreSQL is our application/telemetry database, with pgvector enabled for the RAG/embedding workload.**

That gives us one well-maintained database instead of:

```
SQL Server
SQLite
PostgreSQL
Vector DB
```

We only need:

```
SQL Server       → monitor
PostgreSQL       → store + RAG
```

Much cleaner.

Next step

Let's now create `pg_connection.py`, verify that Python can connect to your local PostgreSQL instance, and then create the PostgreSQL schema for our monitoring telemetry.

We won't touch the SQL Server collectors we've already completed.

Good prect, Here is my exisitng PG connection string which i used in another proejct

DB_CONFIG = {

"host": "localhost",

```
"port": 5432,

"dbname": "gtd_app",
```

Show more

Yes — for this project, I recommend using a `.env` **file**, but with one important rule:

> **Never commit `.env` to GitHub.**

Since your repository is already on GitHub, this is worth setting up correctly now.

Recommended structure

```
AI_SQL_Monitor/
│
├── monitoring/
│   ├── db_connection.py       # SQL Server
│   ├── pg_connection.py       # PostgreSQL
│   └── ...
│
├── .env                       # PostgreSQL password — LOCAL ONLY
├── .gitignore
└── requirements.txt
```

Your `.env` would contain:

```
PG_DB_PASSWORD=mypgdbpassword
```

Then Python reads it using `python-dotenv`.

1. Install `python-dotenv`

From your activated venv:

PowerShell

```
pip install python-dotenv
```

2. Add `.env` to `.gitignore`

Open your project's `.gitignore` and make sure it contains:

```
.env
*.env
```

This is **very important** because your repository is already pushed to GitHub.

3. Create `.env`

At:

```
D:\Python\AI_SQL_Monitor\.env
```

put:

```
PG_DB_PASSWORD=your_actual_postgresql_password
```

Don't put quotes around it unless your password actually requires them.

4. Create `pg_connection.py`

Inside:

```
D:\Python\AI_SQL_Monitor\monitoring\pg_connection.py
```

use:

Python Run

```python
import os
import psycopg2
from dotenv import load_dotenv


load_dotenv()


DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "ai_sql_monitor",
    "user": "postgres",
    "password": os.environ.get("PG_DB_PASSWORD", "")
}


def get_pg_connection():
    return psycopg2.connect(**DB_CONFIG)


if __name__ == "__main__":

    try:
        connection = get_pg_connection()
```

```
        print("Connected to PostgreSQL successfully!")

        connection.close()

    except Exception as e:
        print("PostgreSQL connection failed:")
        print(e)
```

Notice I've changed:

```
gtd_app
```

to:

```
ai_sql_monitor
```

because `gtd_app` belongs to your other project. We should create a **separate PostgreSQL database for this project**.

5. Install PostgreSQL driver

PowerShell

```
pip install psycopg2-binary
```

Then:

PowerShell

```
python pg_connection.py
```

Expected:

```
Connected to PostgreSQL successfully!
```

Environment variables vs `.env`

For **your current local development**, I'd use:

```
.env
   ↓
python-dotenv
   ↓
os.environ
```

```
    ↓
PostgreSQL connection
```

Later, when we deploy to Azure, we **won't copy** `.env` **to production**. We'll use Azure's secret/configuration mechanism instead.

So we're establishing a pattern now that can evolve cleanly:

```
LOCAL DEVELOPMENT
.env
  ↓
PostgreSQL


AZURE PRODUCTION
Azure configuration / managed secret
  ↓
PostgreSQL
```

And one more important point: **don't put your PostgreSQL password directly inside** `pg_connection.py`. Your existing approach of reading it from an environment variable was already the right idea; `.env` simply makes local development more convenient.

Once `pg_connection.py` connects successfully, we'll create the `ai_sql_monitor` **database and our first PostgreSQL telemetry tables**, then modify `collector.py` to persist the snapshot.

psycopg2
i have having erorr. I have already installed it using pip

(venv) PS D:\Python\AI_SQL_Monitor\monitoring>
python.exe .\pg_connection.py
Traceback (most recent call last):
  File
"D:\Python\AI_SQL_Monitor\monitoring\pg_connection.py"
, line 2, in <module>
    import psycopg2
ModuleNotFoundError: No module named 'psycopg2'

Yes — this is the same kind of issue we had with `pyodbc` earlier: `pip` **and** `python` **are pointing to different environments**.

Your prompt says `(venv)`, but we've already seen that your `python` command was being intercepted by the Windows Store Python alias. Let's verify the environment directly rather than reinstalling blindly.

1. Run these three commands

From:

```
D:\Python\AI_SQL_Monitor\monitoring
```

run:

```
PowerShell
.\venv\Scripts\python.exe -m pip --version
```

Then:

```
PowerShell
.\venv\Scripts\python.exe -m pip show psycopg2-binary
```

Then:

```
PowerShell
.\venv\Scripts\python.exe -c "import psycopg2; print(psycopg2.__version__)"
```

2. If the second command says package not found

Install it explicitly into **this exact venv**:

```
PowerShell
.\venv\Scripts\python.exe -m pip install psycopg2-binary
```

Then verify:

```
PowerShell
.\venv\Scripts\python.exe -c "import psycopg2; print(psycopg2.__version__)"
```

You should get something similar to:

```
2.9.x
```

3. Then run your connection program using the same interpreter

Instead of:

```
PowerShell
python.exe .\pg_connection.py
```

use:

```
PowerShell
.\venv\Scripts\python.exe .\pg_connection.py
```

This avoids the Windows `python` alias problem completely.

One important point

Earlier you showed:

```
(venv)
python -c "import sys; print(sys.executable)"
```

returning:

```
C:\Users\Moosa\AppData\Local\Microsoft\WindowsApps\Python...
```

That means your **PowerShell `python` command was not actually using the venv**, even though `(venv)` appeared in the prompt.

So from now on, while we're sorting this out, I'd use:

```
PowerShell
.\venv\Scripts\python.exe
```

and:

```
PowerShell
.\venv\Scripts\python.exe -m pip
```

They are guaranteed to refer to the same Python installation.

Once `psycopg2` imports successfully, we can continue with the PostgreSQL connection.

venv) PS D:\Python\AI_SQL_Monitor>
.\venv\Scripts\python.exe -m pip --version
pip 24.0 from D:\Python\AI_SQL_Monitor\venv\Lib\site-packages\pip (python 3.11)
(venv) PS D:\Python\AI_SQL_Monitor>
.\venv\Scripts\python.exe -m pip show psycopg2-binary
WARNING: Package(s) not found: psycopg2-binary
(venv) PS D:\Python\AI_SQL_Monitor>

Exactly — we found the issue. 👍

Your venv is healthy:

```
pip 24.0
D:\Python\AI_SQL_Monitor\venv\Lib\site-packages
```

but `psycopg2-binary` **is simply not installed in that venv**. The earlier installation was likely done against a different Python environment.

Install it into the correct venv

Run exactly:

PowerShell

```
.\venv\Scripts\python.exe -m pip install psycopg2-binary
```

Then verify:

PowerShell

```
.\venv\Scripts\python.exe -m pip show psycopg2-binary
```

And finally:

PowerShell

```
.\venv\Scripts\python.exe -c "import psycopg2; print(psycopg2.__version__)"
```

You should see the installed version.

Then test our connection

From the project root:

```
PowerShell

cd D:\Python\AI_SQL_Monitor
```

and run:

```
PowerShell

.\venv\Scripts\python.exe .\monitoring\pg_connection.py
```

Expected:

```
Connected to PostgreSQL successfully!
```

Don't reinstall Python or recreate the venv. Your existing venv is fine; we just need to install the package into the interpreter we're actually using.

Once that connects, we'll move directly to creating the PostgreSQL telemetry schema.

(venv) PS D:\Python\AI_SQL_Monitor\monitoring> python.exe .\pg_connection.py
PostgreSQL connection failed:
connection to server at "localhost" (::1), port 5432 failed: FATAL: database "ai_sql_monitor" does not exist

we did't created this db in the PG yet. we need to script do that. However so far the connection is working i guess.

Also please give me how many token i can left in each reply. so that i will come to when i will max out?

Yes — **the PostgreSQL connection itself is working**. The error is actually good news:

```
FATAL: database "ai_sql_monitor" does not exist
```

This means Python successfully reached:

```
localhost:5432
```

and PostgreSQL responded. We just haven't created the project database yet.

1. Let's create the database properly

Rather than manually creating it in pgAdmin, let's make a small bootstrap script. This is better for our Git project because the setup becomes reproducible.

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\create_pg_database.py
```

Use:

Python Run

```python
import os
import psycopg2
from dotenv import load_dotenv


load_dotenv()


DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "postgres",
    "user": "postgres",
    "password": os.environ.get("PG_DB_PASSWORD", "")
}

TARGET_DATABASE = "ai_sql_monitor"


def create_database():
    connection = psycopg2.connect(**DB_CONFIG)

    try:
        connection.autocommit = True

        cursor = connection.cursor()

        cursor.execute(
            "SELECT 1 FROM pg_database WHERE datname = %s",
            (TARGET_DATABASE,)
        )

        exists = cursor.fetchone()

        if exists:
            print(f"Database '{TARGET_DATABASE}' already exists.")
        else:
```

```
            cursor.execute(
                f'CREATE DATABASE "{TARGET_DATABASE}"'
            )
            print(f"Database '{TARGET_DATABASE}' created successfully.")

        cursor.close()

    finally:
        connection.close()


if __name__ == "__main__":
    try:
        create_database()
    except Exception as e:
        print("Database creation failed:")
        print(e)
```

Why do we connect to `postgres` first?

This is important.

You **cannot connect to `ai_sql_monitor` to create `ai_sql_monitor`** if it doesn't exist.

So:

```
PostgreSQL server
      |
      ▼
postgres database
      |
      └── CREATE DATABASE ai_sql_monitor
```

Then our application connects to:

```
ai_sql_monitor
```

---

2. Run it

From your project root:

```
PowerShell

cd D:\Python\AI_SQL_Monitor
```

Then:

PowerShell

```
.\venv\Scripts\python.exe .\monitoring\create_pg_database.py
```

Expected:

```
Database 'ai_sql_monitor' created successfully.
```

If you run it again:

```
Database 'ai_sql_monitor' already exists.
```

That's intentional—the script is safe to run repeatedly.

---

3. Now test `pg_connection.py`

Run:

PowerShell

```
.\venv\Scripts\python.exe .\monitoring\pg_connection.py
```

You should now get:

```
Connected to PostgreSQL successfully!
```

At that point we'll have:

```
SQL Server
localhost
   |
   └── Vehicle
       └── monitored database


PostgreSQL
localhost:5432
   |
   └── ai_sql_monitor
       └── our monitoring platform
```

About your token question

I **can't see or reliably report your remaining token/usage quota** for the conversation. The usage-limit behavior is controlled by the ChatGPT product, and I don't receive a live

"X tokens remaining" counter.

So there isn't a reliable number I can give you like:

> "You have 18,000 tokens left."

If the interface shows a usage/reset indicator, **that's the authoritative source**.

Also, you don't need to wait until you're completely maxed out. If the conversation eventually becomes too long or a usage limit interrupts it, we can continue from a concise project-state summary. I'll keep the project architecture and decisions organized as we go.

For now, **run** `create_pg_database.py` **and then** `pg_connection.py`. If both succeed, we'll create the PostgreSQL schema for telemetry next.

Create the PostgreSQL schema for telemetry

Yes. Now that `ai_sql_monitor` exists and the connection works, let's create the **first proper PostgreSQL telemetry schema**.

I recommend we start with **one snapshot table**, not dozens of metric tables. Your `collector.py` already produces a complete snapshot, so PostgreSQL can initially store that snapshot as structured JSONB while we evolve the schema.

Target design

```
PostgreSQL
└── ai_sql_monitor
    │
    ├── monitoring_snapshots
    │       ├── id
    │       ├── collected_at
    │       ├── server_name
    │       ├── database_name
    │       └── snapshot
    │
    └── later...
            ├── alerts
            ├── incidents
            ├── rules
            └── RAG tables + pgvector
```

This gives us flexibility while we're still developing the monitoring model.

1. Install nothing else

`psycopg2-binary` and `python-dotenv` are already what we need.

2. Create `create_pg_schema.py`

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\create_pg_schema.py
```

Use:

Python Run

```python
import os
import psycopg2
from dotenv import load_dotenv


load_dotenv()


DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "ai_sql_monitor",
    "user": "postgres",
    "password": os.environ.get("PG_DB_PASSWORD", "")
}


def create_schema():
    connection = psycopg2.connect(**DB_CONFIG)

    try:
        cursor = connection.cursor()

        cursor.execute("""
            CREATE TABLE IF NOT EXISTS monitoring_snapshots (
                id BIGSERIAL PRIMARY KEY,

                collected_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

                server_name VARCHAR(255),

                database_name VARCHAR(255),

                snapshot JSONB NOT NULL
            );
        """)
```

```
        cursor.execute("""
            CREATE INDEX IF NOT EXISTS idx_monitoring_snapshots_collected_at
            ON monitoring_snapshots (collected_at);
        """)

        cursor.execute("""
            CREATE INDEX IF NOT EXISTS idx_monitoring_snapshots_server
            ON monitoring_snapshots (server_name);
        """)

        cursor.execute("""
            CREATE INDEX IF NOT EXISTS idx_monitoring_snapshots_snapshot
            ON monitoring_snapshots
            USING GIN (snapshot);
        """)

        connection.commit()

        print("PostgreSQL telemetry schema created successfully.")

    finally:
        connection.close()


if __name__ == "__main__":

    try:
        create_schema()

    except Exception as e:
        print("Schema creation failed:")
        print(e)
```

3. Run it

From:

```
D:\Python\AI_SQL_Monitor
```

run:

PowerShell

```
.\venv\Scripts\python.exe .\monitoring\create_pg_schema.py
```

Expected:

```
PostgreSQL telemetry schema created successfully.
```

Why JSONB for the snapshot?

This is deliberate.

Our current collector contains different structures:

```
CPU
Memory
Sessions
Requests
Blocking
Locks
Server health
```

Some are single objects:

JSON

```
{
    "metric": "memory",
    "total_physical_memory_gb": 31.75
}
```

while others are collections:

JSON

```
{
    "metric": "blocking",
    "count": 2,
    "blocking_requests": [...]
}
```

Trying to normalize everything into 15–20 PostgreSQL tables **right now** would slow down development and lock us into a schema before we've finished defining the monitoring model.

Instead:

```
collector.py
      ↓
complete Python dictionary
      ↓
JSONB
      ↓
PostgreSQL
```

PostgreSQL's `JSONB` still allows us to query inside the snapshot and index it.

Later, once we understand which metrics need high-frequency analytical queries, we can introduce dedicated tables for those metrics.

One important design decision

I'm intentionally **not enabling pgvector yet**.

`pgvector` belongs to our RAG layer:

```
Monitoring telemetry
        ↓
PostgreSQL JSONB/tables


SQL Server documentation
DBA knowledge
Runbooks
Known problems
Solutions
        ↓
PostgreSQL + pgvector
        ↓
RAG
        ↓
LLM
```

We don't need embeddings for:

```
CPU = 87%
Blocking = 2
Memory = 75%
```

Those are structured telemetry.

We need vectors for knowledge such as:

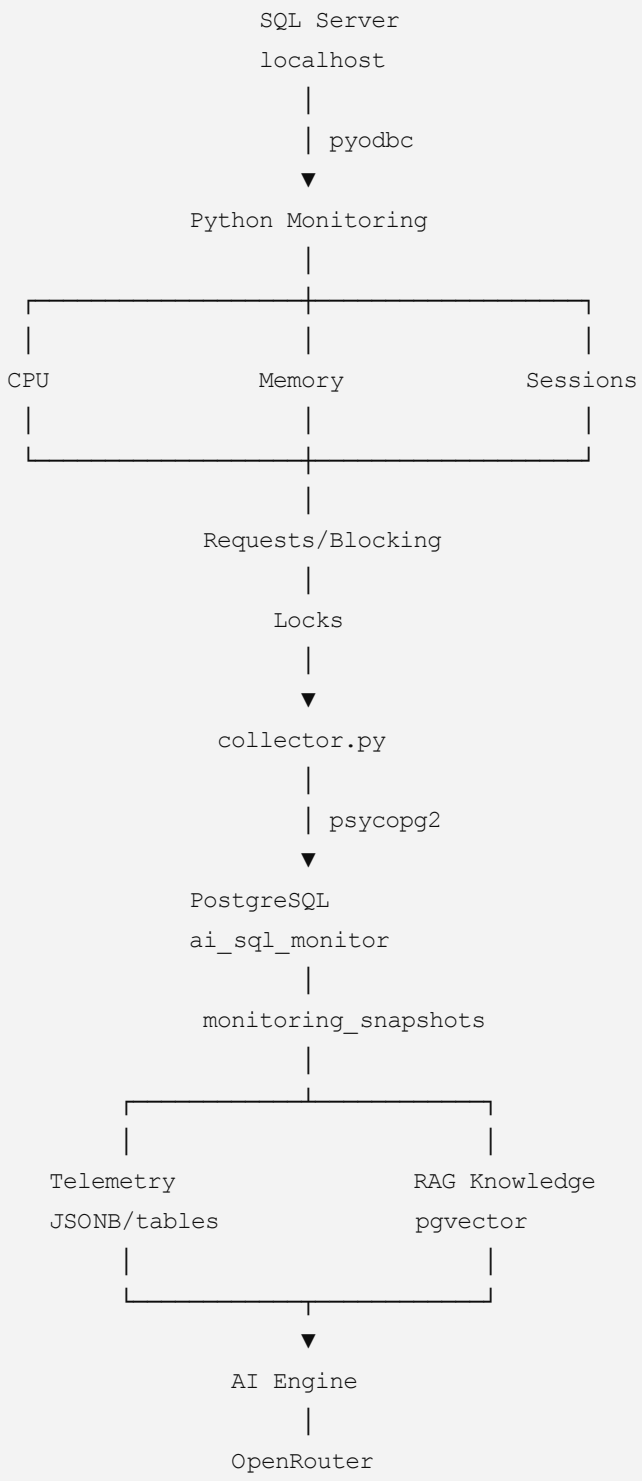
> "What does LCK_M_IS indicate?"

> "What are common causes of long-running blocking transactions?"

> "How should a DBA investigate excessive PAGEIOLATCH waits?"

That separation will make the AI architecture much cleaner.

Our architecture now



Next step

Once `create_pg_schema.py` reports success, we'll modify `collector.py` **to automatically save every snapshot into PostgreSQL**.

Then we will have our first real end-to-end pipeline:

SQL Server → Python → PostgreSQL

After that, we'll build the scheduler so this happens continuously rather than only when you manually execute `collector.py`.

done, please proceed next

Great. Now we're at an important point: **the PostgreSQL database and telemetry schema exist**.

Next we'll connect `collector.py` to PostgreSQL so every monitoring snapshot can be persisted.

Step 7 — Save snapshots to PostgreSQL

We'll keep the responsibilities separate:

```
db_connection.py
    ↓
SQL Server connection

pg_connection.py
    ↓
PostgreSQL connection

collector.py
    ↓
Collect SQL Server telemetry
    ↓
Save snapshot to PostgreSQL
```

1. Update `pg_connection.py`

Let's make the PostgreSQL connection reusable by other Python modules.

Your `pg_connection.py` should look like this:

Python Run

```python
import os
import psycopg2
from dotenv import load_dotenv


load_dotenv()
```

```python
DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "dbname": "ai_sql_monitor",
    "user": "postgres",
    "password": os.environ.get("PG_DB_PASSWORD", "")
}


def get_pg_connection():
    return psycopg2.connect(**DB_CONFIG)


if __name__ == "__main__":

    try:
        connection = get_pg_connection()

        print("Connected to PostgreSQL successfully!")

        connection.close()

    except Exception as e:
        print("PostgreSQL connection failed:")
        print(e)
```

---

2. Create `telemetry_repository.py`

Instead of putting PostgreSQL INSERT statements directly inside `collector.py`, create a repository.

Create:

```
D:\Python\AI_SQL_Monitor\monitoring\telemetry_repository.py
```

Use:

Python Run

```python
from pg_connection import get_pg_connection
import json


def save_snapshot(snapshot):

    connection = get_pg_connection()
```

```
    try:
        cursor = connection.cursor()

        server_name = snapshot.get("server_health", {}).get("server")
        database_name = snapshot.get("server_health", {}).get("database")

        cursor.execute(
            """
            INSERT INTO monitoring_snapshots
            (
                server_name,
                database_name,
                snapshot
            )
            VALUES (%s, %s, %s::jsonb)
            """,
            (
                server_name,
                database_name,
                json.dumps(snapshot, default=str)
            )
        )

        connection.commit()

    finally:
        connection.close()


if __name__ == "__main__":
    print("Telemetry repository module loaded successfully.")
```

Notice something important:

Python Run

```
json.dumps(snapshot, default=str)
```

Our snapshot contains things such as SQL Server datetime values. `default=str` makes sure those Python objects can be serialized into JSON.

---

3. Update `collector.py`

Now modify your existing `collector.py`.

At the top, add:

Python Run

```python
from telemetry_repository import save_snapshot
```

Then change:

Python Run

```python
if __name__ == "__main__":
```

to:

Python Run

```python
if __name__ == "__main__":

    snapshot = collect_snapshot()

    save_snapshot(snapshot)

    print("\nMonitoring snapshot collected and saved to PostgreSQL.\n")

    print(snapshot)
```

So your **complete** `collector.py` should now be:

Python Run

```python
from server_health import get_server_health
from cpu import get_cpu_usage
from memory import get_memory_status
from sessions import get_user_sessions
from requests import get_active_requests
from blocking import get_blocking_requests
from locks import get_locks

from telemetry_repository import save_snapshot


def collect_snapshot():

    snapshot = {
        "server_health": get_server_health(),
        "cpu": get_cpu_usage(),
        "memory": get_memory_status(),
        "sessions": get_user_sessions(),
        "requests": get_active_requests(),
        "blocking": get_blocking_requests(),
        "locks": get_locks()
    }

    return snapshot
```

```
if __name__ == "__main__":
    snapshot = collect_snapshot()
    save_snapshot(snapshot)
    print("\nMonitoring snapshot collected and saved to PostgreSQL.\n")
    print(snapshot)
```

---

4. Run it

From:

```
D:\Python\AI_SQL_Monitor\monitoring
```

run:

```
PowerShell
python collector.py
```

You should still see all your existing output, but now you'll also see:

```
Monitoring snapshot collected and saved to PostgreSQL.
```

---

5. Verify directly in PostgreSQL

Open pgAdmin or your PostgreSQL client and connect to:

```
ai_sql_monitor
```

Run:

```
SQL
SELECT
    id,
    collected_at,
    server_name,
    database_name
```

```
FROM monitoring_snapshots
```

You should see something like:

```
id    collected_at                server_name        database_name
1     2026-09-10 10:...            Moosa-DELLLaptop   Vehicle
```

Then try:

SQL

```sql
SELECT
    id,
    collected_at,
    snapshot
FROM monitoring_snapshots
ORDER BY id DESC
LIMIT 1;
```

You should see the entire monitoring snapshot stored as JSONB.

🎯 This is our first major end-to-end milestone

We now have:

```
            SQL Server
                |
                | pyodbc
                ▼
        Python collectors
                |
                ▼
          collector.py
                |
                | complete snapshot
                ▼
           PostgreSQL
                |
                ▼
      monitoring_snapshots
```

And importantly, **we haven't introduced the LLM yet**.

That's intentional.

The next stage after this is to make the collector run continuously:

```
10:00:00 → collect → save
10:00:30 → collect → save
10:01:00 → collect → save
10:01:30 → collect → save
...
```

But before we build the scheduler, we should make sure the **snapshot persistence works correctly**.

Run:

PowerShell

```
python collector.py
```

Then check:

SQL

```
SELECT COUNT(*)
FROM monitoring_snapshots;
```

If it returns `1` (or increases by one each time you run the collector), we're ready for the continuous monitoring loop.

ChatGPT can make mistakes. Check important info.